



10-x23 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Course Overview + RNN-LMs + Automatic Differentiation

Matt Gormley & Aran Nayebi
Lecture 1

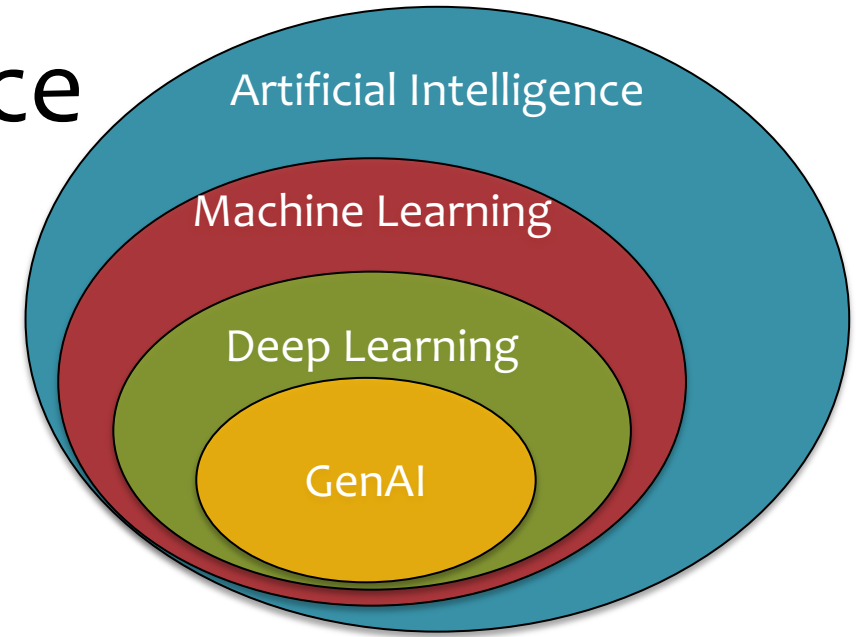
WHAT IS GENERATIVE AI?

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

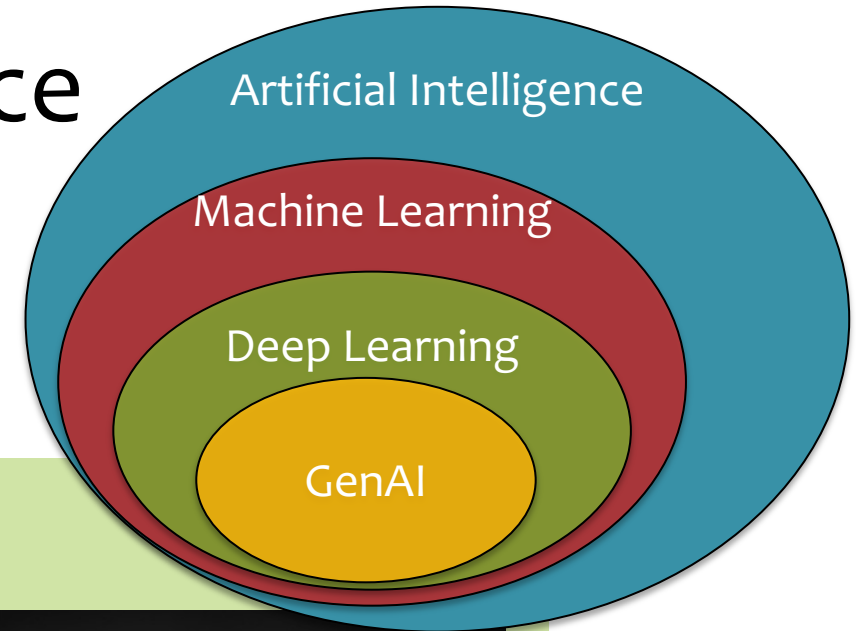


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

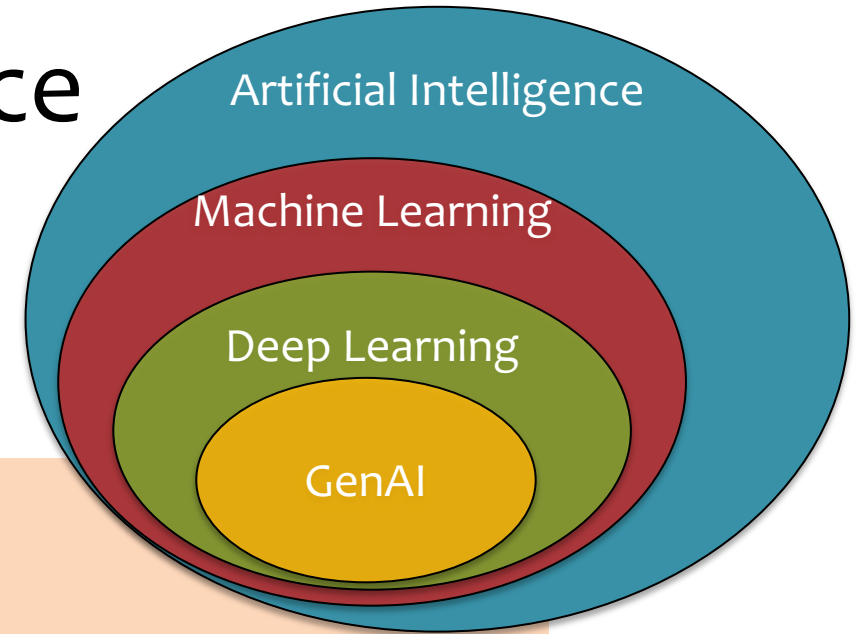


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

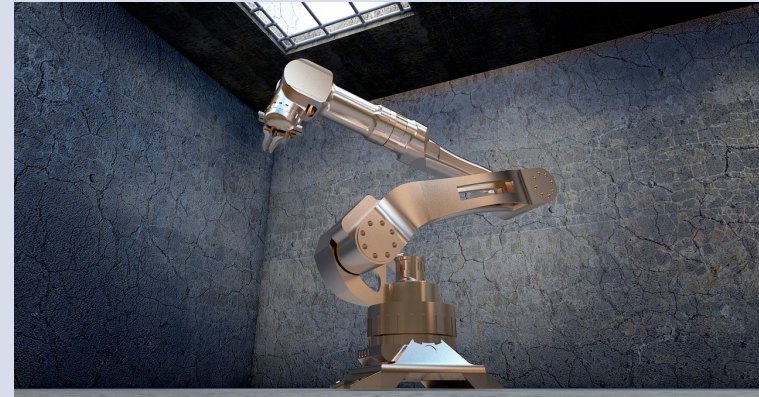
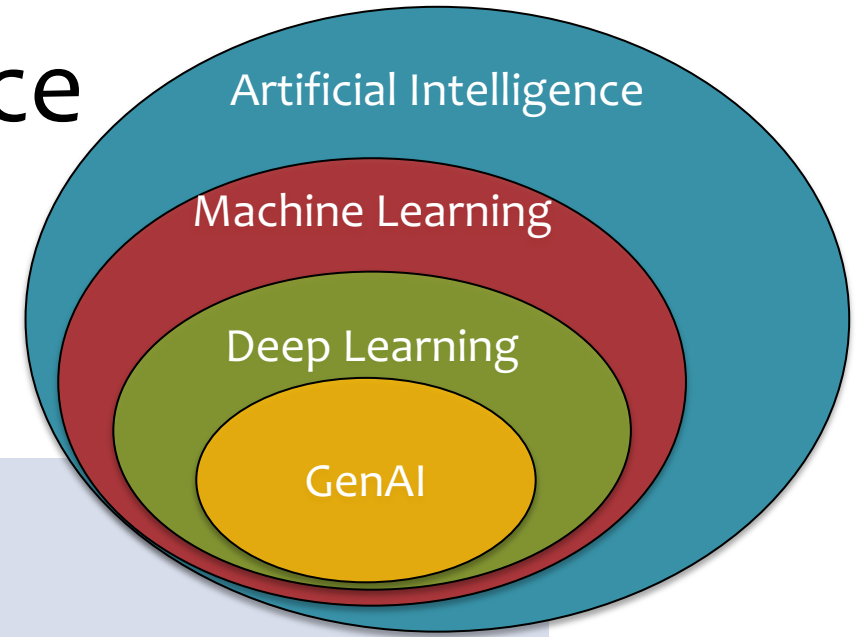


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

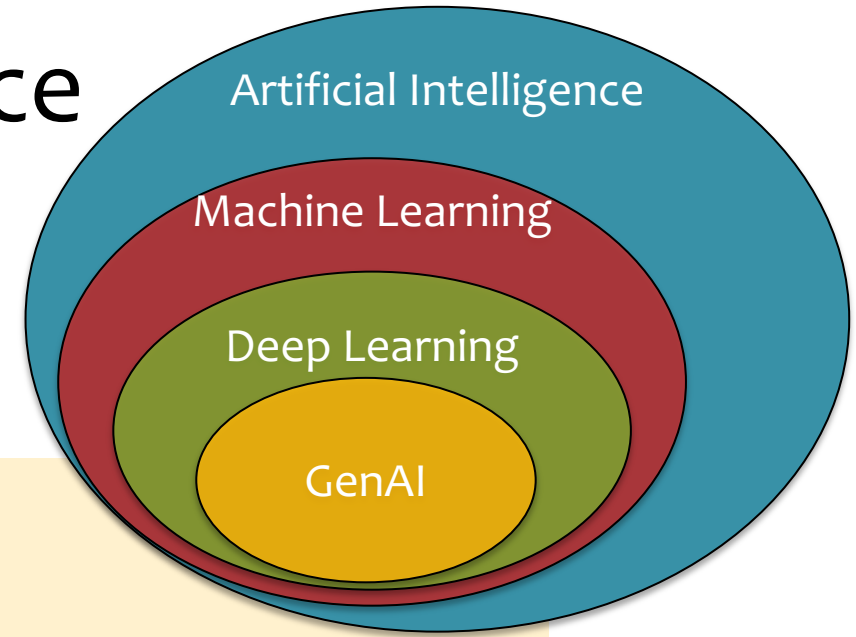


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

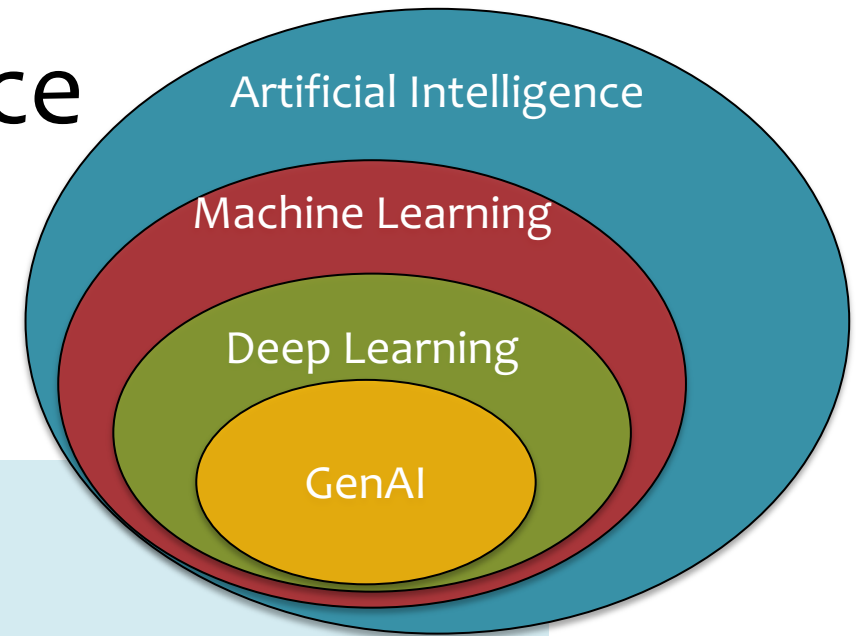


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

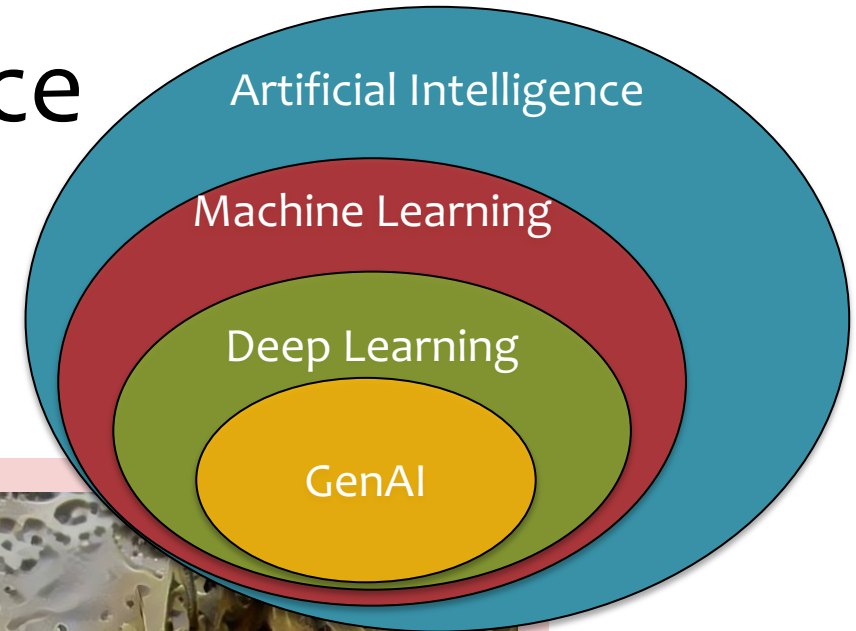


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

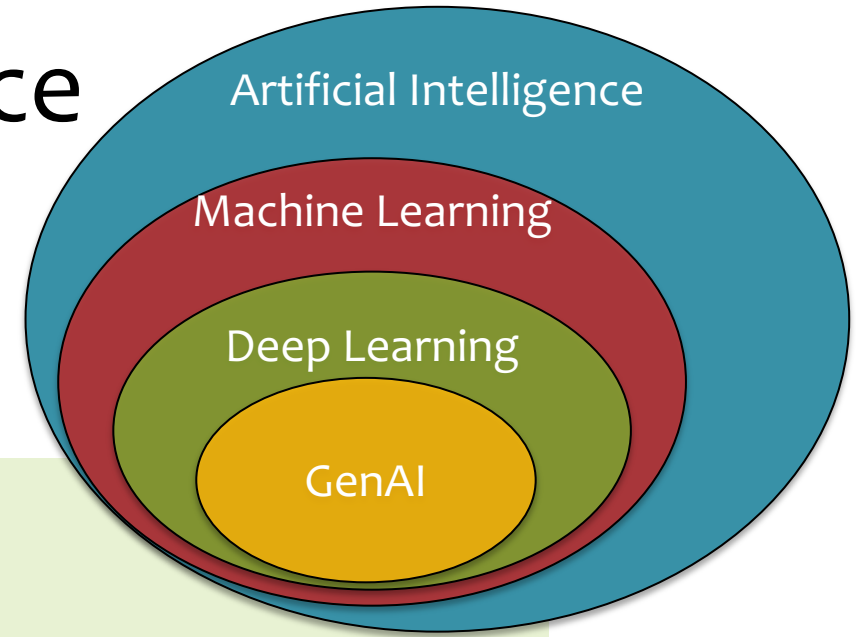


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning

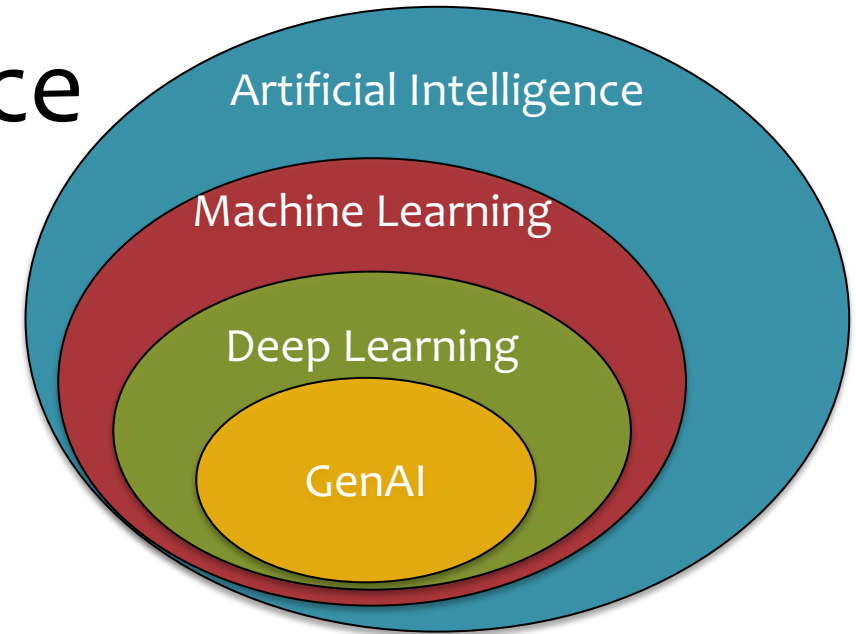


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



❑ Q: What does Generative AI have to do with **any of these goals?**

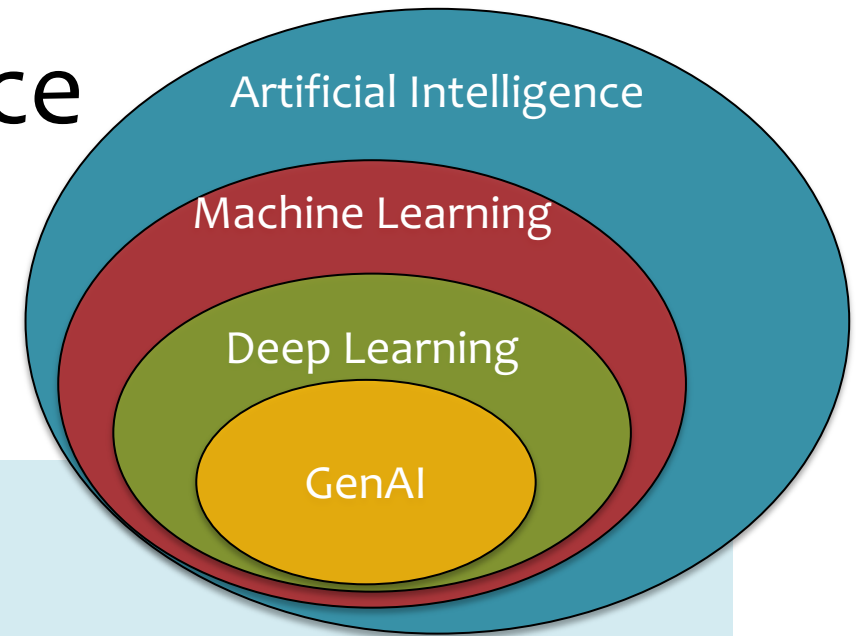
❑ A: It's making in-roads into **all of them.**

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



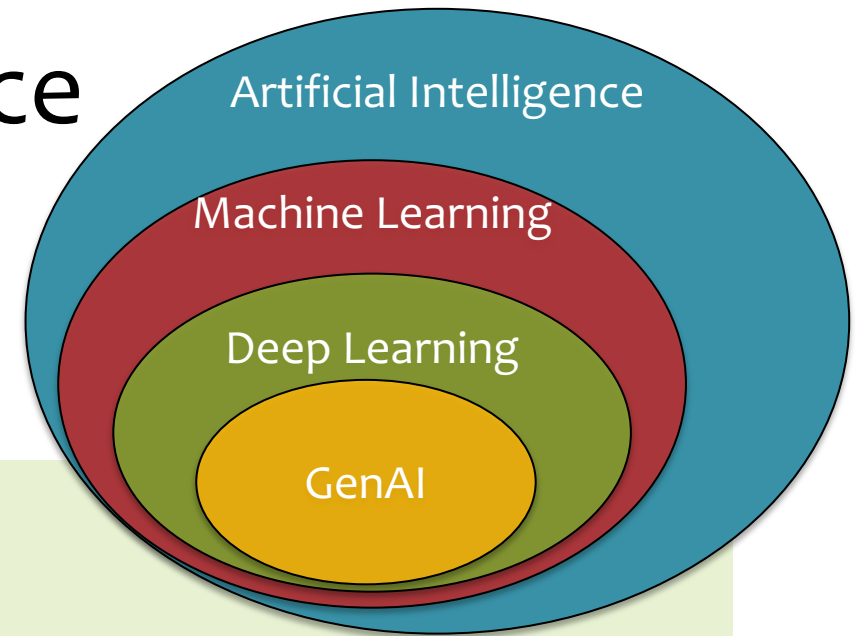
- ☐ Communication comprises the comprehension and generation of human language.
- ☐ Large language models (LLMs) excel at both
- ☐ (Even though they are most often trained autoregressively, i.e. to **generate** a next word, given the previous ones)

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



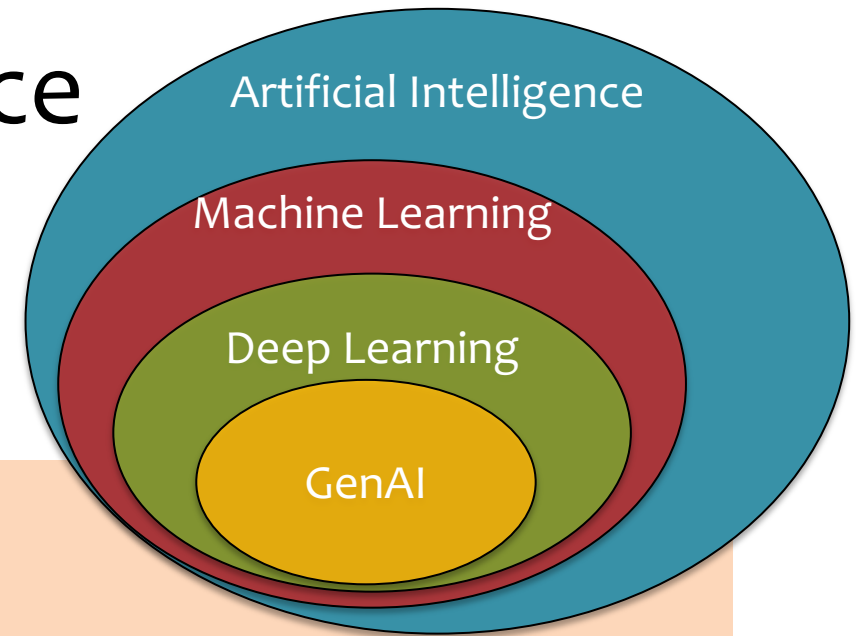
- ❑ The traditional way of learning in ML is via **parameter estimation**
- ❑ But **in-context learning** (i.e. providing training examples as context at test time) shows that learning can also be done via **inference**

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



- ❑ LLMs are also (unexpectedly) good at certain reasoning tasks
- ❑ cf. Chain-of-Thought Prompting (an ex. of in-context learning)

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

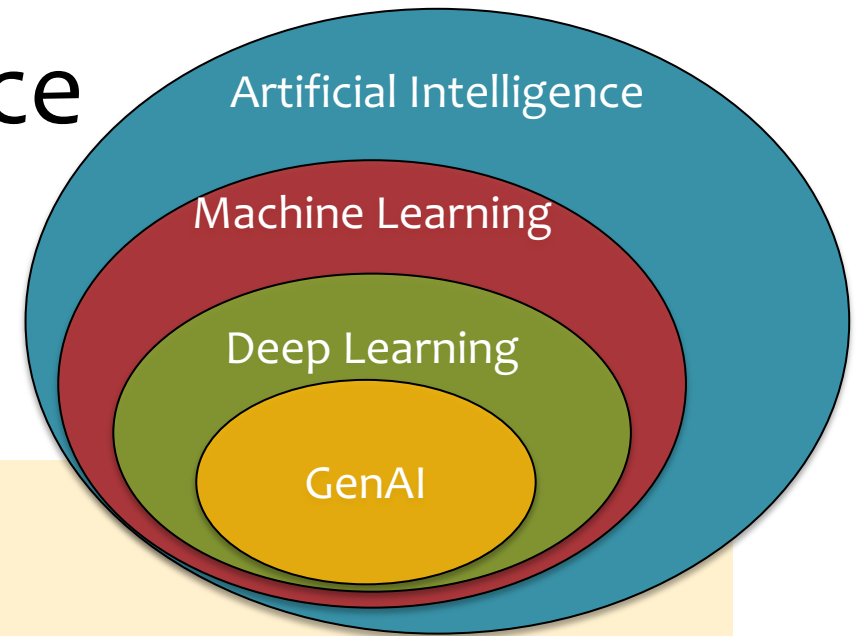
A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



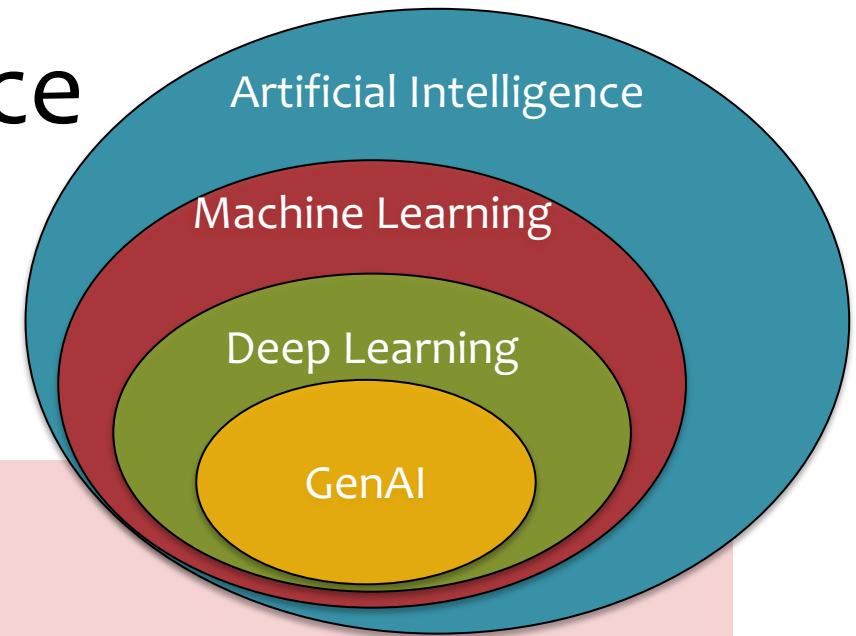
- ❑ LLMs are already being used for grounded planning for embodied agents, c.f. LLM-Planner
- ❑ Planning is a key step for agentic code assistants

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



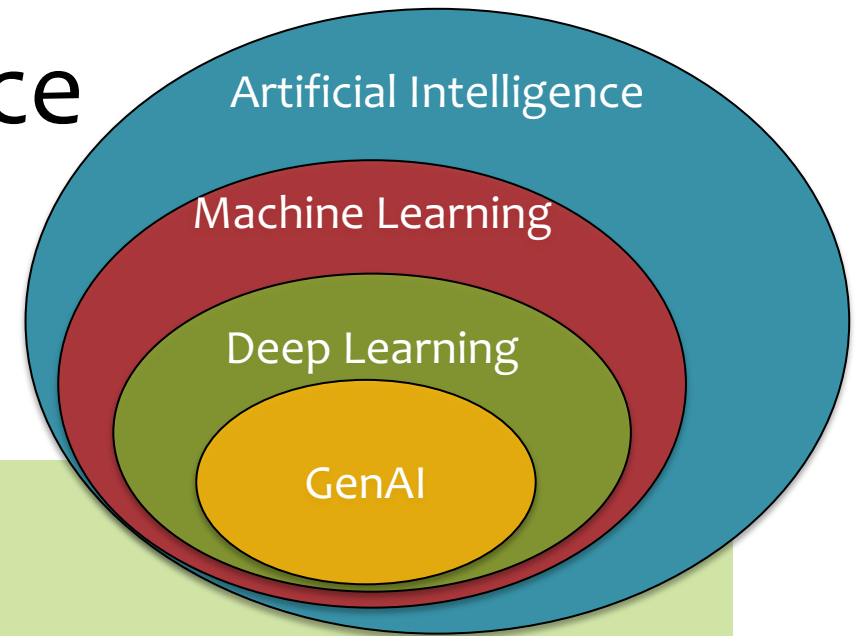
- ❑ Text-to-image models
[Midjourney's Discord server has 18 million members (1.7 million were online this morning)]
- ❑ Text-to-music models
[MusicGen capable of conditioning on text and audio sample]

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



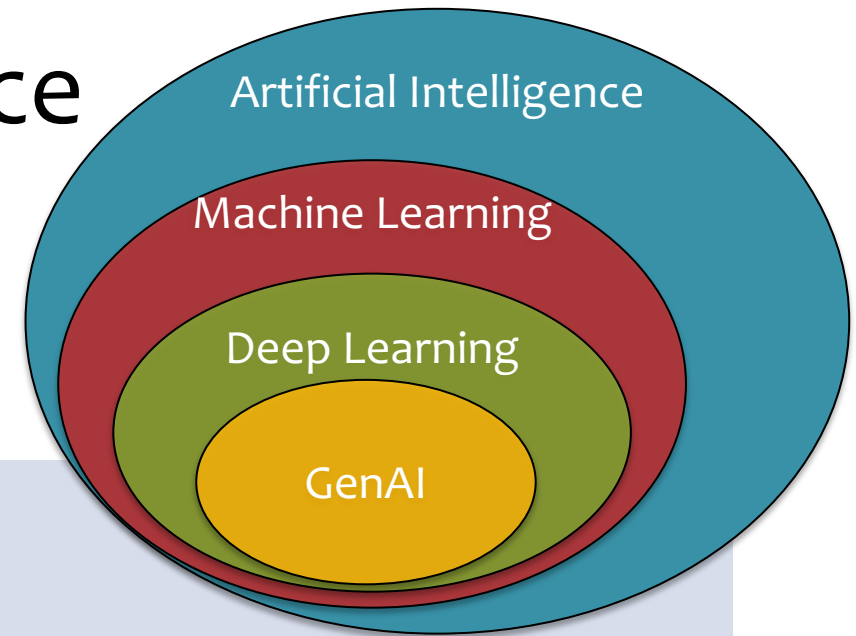
- ❑ Multimodal foundation models learn to answer questions about images (and text in images)
- ❑ Diffusion models can be used as zero-shot classifiers

Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

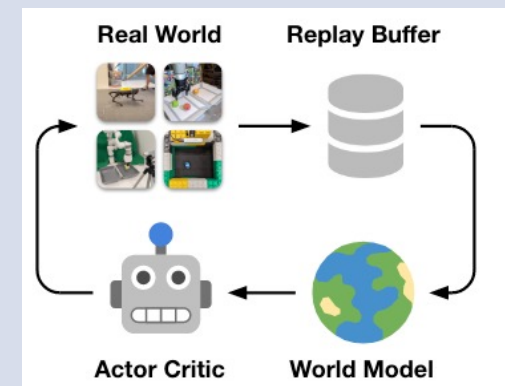
This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



❑ DayDreamer learns a generative model of experiences for RL, i.e. a World Model, without simulation

❑ Quadruped robot learns to walk in under 1 hour

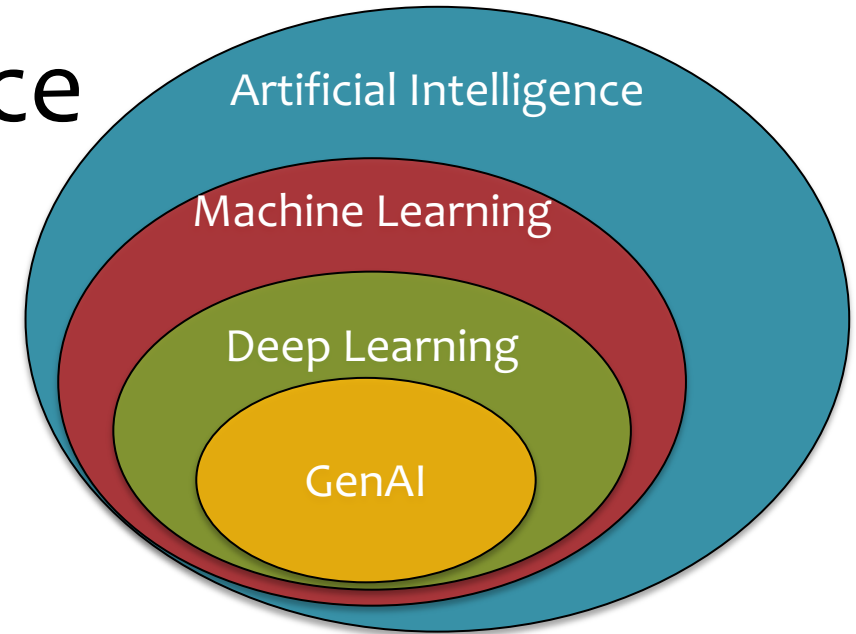


Artificial Intelligence

The basic goal of AI is to develop intelligent machines.

This consists of many sub-goals:

- Perception
- Reasoning
- Control / Motion / Manipulation
- Planning
- Communication
- Creativity
- Learning



❑ Q: What does Generative AI have to do with **any of these goals?**

❑ A: It's making in-roads into **all of them.**

WHAT IS GENERATIVE AI CAPABLE OF?

Text Generation

Prompt: Write a proof of the fact that there are infinitely many primes; do it in the style of a Shakespeare play through a dialogue between two parties arguing over the proof.

GPT-4: Enter FIRST PARTY and SECOND PARTY.

FIRST PARTY: Thou dost not understand, my friend,
There simply is no upper end To primes, those
numbers indivisible, No matter if it seems impossible.

SECOND PARTY: But how dost thou so surely know?
To prove this truth, what canst thou show?

FIRST PARTY: Consider this, my doubtful peer, A
clever proof that will make clear: Assume that there's
a final prime, The largest one we'll see in time. Let's
call this number "P" for short, And then, my friend,
we will purport To build a number "Q" that's new,
One larger than "P" and prime too.

SECOND PARTY: But how, my friend, will this be
done? The proof is not yet fairly won.

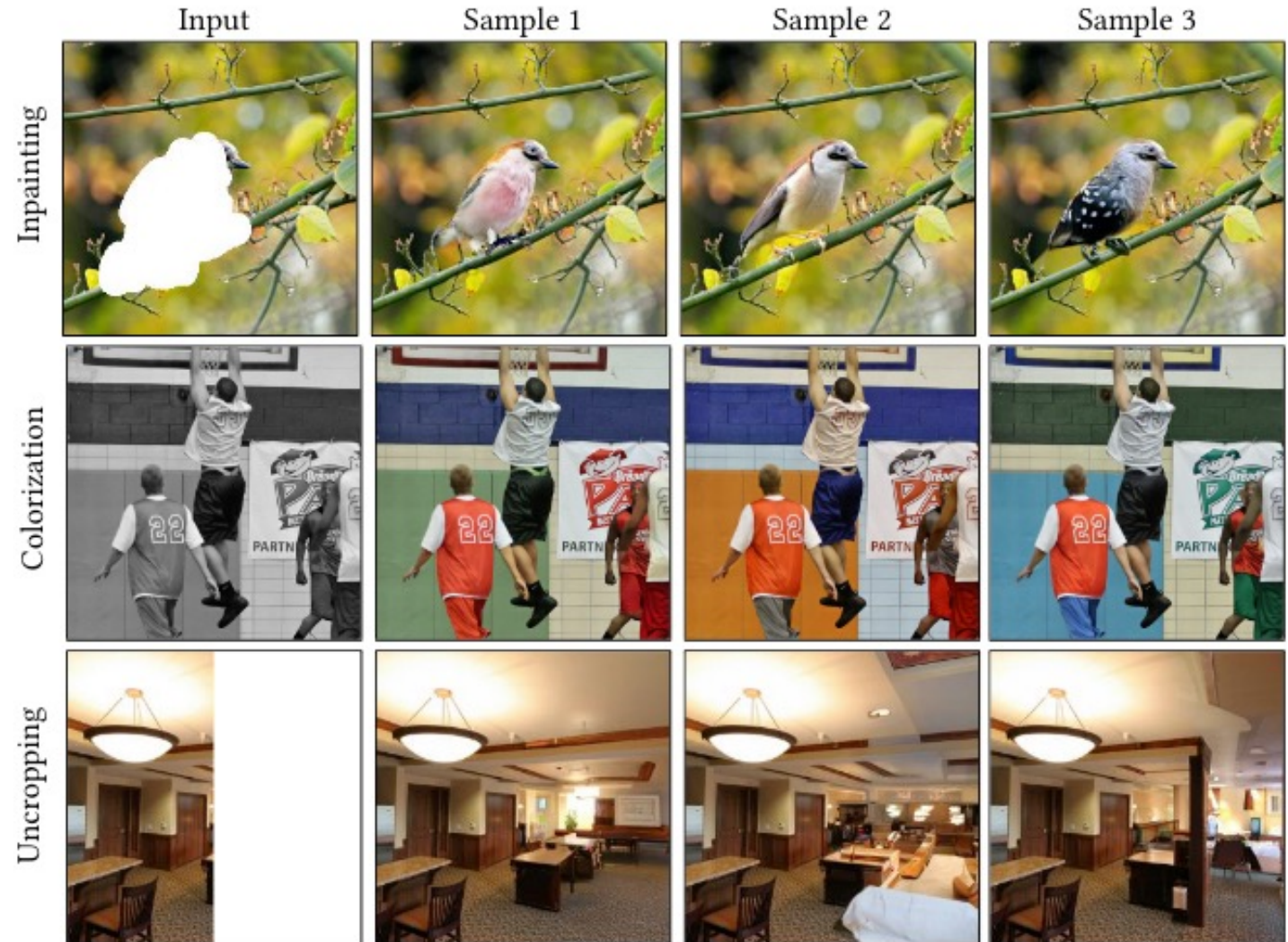
FIRST PARTY: It's simple, friend, just wait and see,
We'll multiply the primes, that's key. All primes from
two to "P" combined, Will make this new "Q" we will
find. But one thing more we must include, To make
the proof more watertight, We'll add a one onto the
end, And now the proof is in our sight.

SECOND PARTY: I see it now, it's crystal clear, For "Q"
must be a prime, my dear. It can't divide by any prime,
All smaller than itself in time. And since we chose "P"
as the last, This proves that our assumption's past.
There is no final prime, we see, The numbers go on
endlessly.

Image Editing

A variety of tasks involve automatic editing of an image:

- **Inpainting** fills in the (pre-specified) missing pixels
- **Colorization** restores color to a greyscale image
- **Uncropping** creates a photo-realistic reconstruction of a missing side of an image



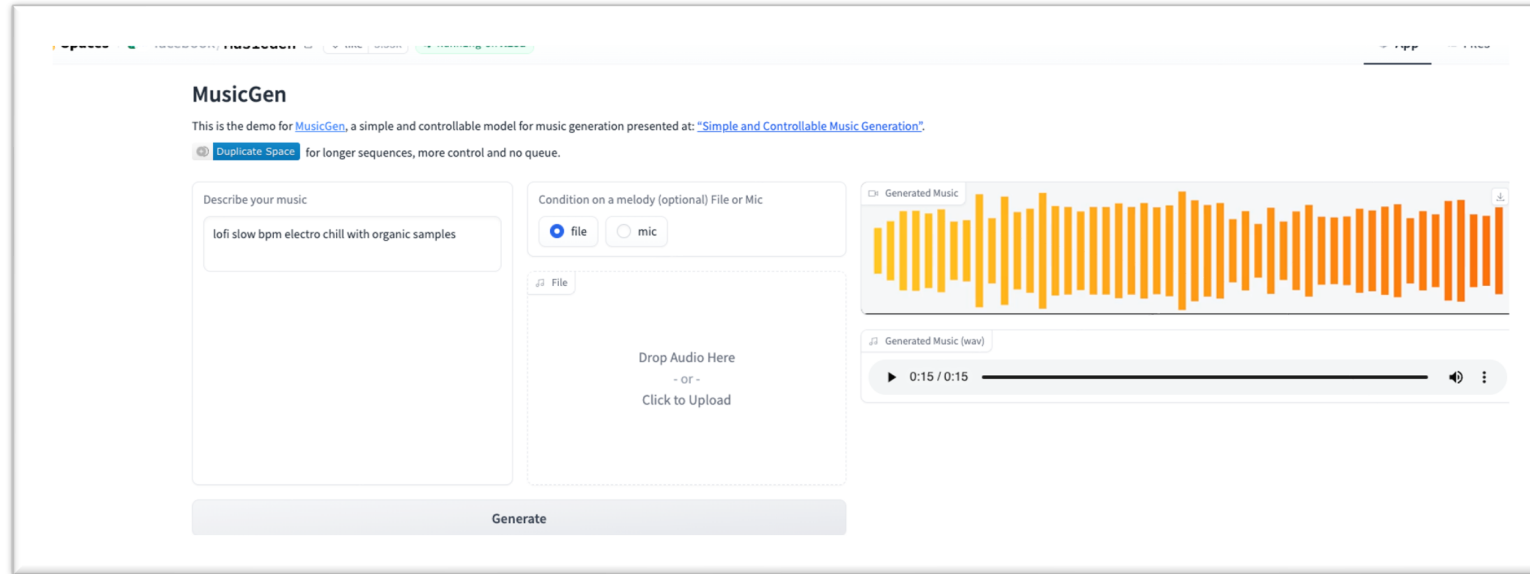
Text-to-Image Generation

- Given a text description, sample an image that depicts the prompt
- The following images are samples from SDXL with refinement

Prompt: close up headshot, futuristic **old man**, wild hair sly smile in front of gigantic UFO, dslr, sharp focus, dynamic composition, **rule of thirds**

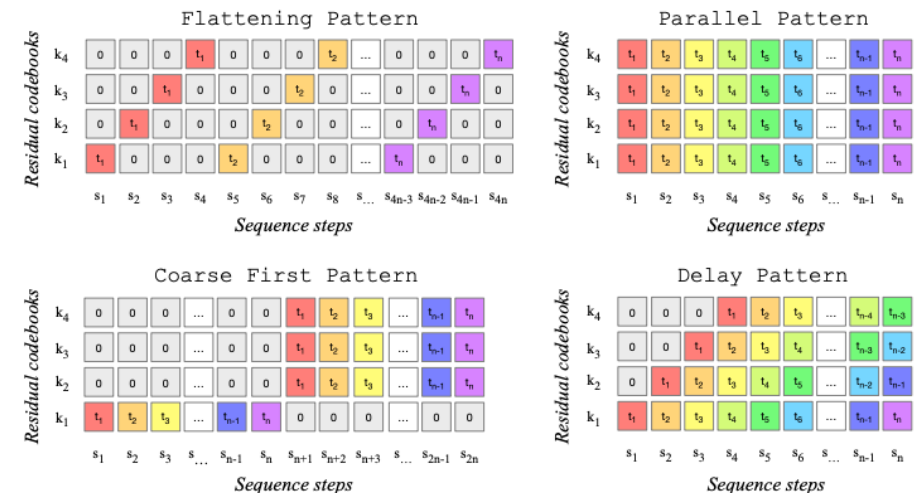


Music Generation



MusicGen

- A transformer decoder model over quantized units (discrete elements of a codebook of audio frames)
- Interleaves sounds by adjusting how codebooks attend to each other
- Permits conditioning on text and/or audio samples



Code Generation



Figure 1.3: We queried GPT-4 three times, at roughly equal time intervals over the span of a month while the system was being refined, with the prompt “Draw a unicorn in TikZ”. We can see a clear evolution in the sophistication of GPT-4’s drawings.

GPT-4 vs ChatGPT

Prompt:

A function f is called *increasing* if $f(s) \leq f(t)$ holds for every $s \leq t$. Consider the formula:
 $\lim_{x \rightarrow 0} \lim_{n \rightarrow \infty} \left[\sum_{i=1}^n (f(nx) - \int_0^x \sin(nx) dx), \frac{(x + \frac{2}{8})^2 - 5}{4x - 1/x^2}, \dots, \text{partial } f(x/n) \right]$
 over partial $x * (e^{-i\omega n x} + 3(x \# 7))$, f inverse of $(x \bmod n)$]
 multiply $\text{matrix}[1, 1, \dots, 1 \setminus 1, 1, \dots, 1]$
 {grey_color{this gives us a good impression about the function}}

Convert the above paragraph to LaTeX.

GPT-4:

A function f is called *increasing* if $f(s) \leq f(t)$ holds for every $s \leq t$. Consider the formula:

$$\lim_{x \rightarrow 0} \lim_{n \rightarrow \infty} \left[\sum_{i=1}^n (f(nx) - \int_0^x \sin(nx) dx), \frac{(x + \frac{2}{8})^2 - 5}{4x - 1/x^2}, \dots, \text{partial } f(x/n) \right] \times \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

this gives us a good impression about the function

GPT-4 vs ChatGPT

Prompt:

I want to write a customized optimizer in pytorch, the optimizer should do:
 1. the parameters for the optimizer is learning rate, momentum, weight decay, d_dim, k and factor alpha
 2. the optimizer goes through each parameter in the network, and then
 -- Get the gradient of this parameter, reshape it to 2D by looping through the dimensions, and keeping the dimension whose actual dimension (shape) is equal to d_dim, do an SVD decomposition of the parameter so $W = U \Sigma V$.
 -- Keep the top k singular vectors of W , so we get a new matrix $W_k = U \Sigma_k V$.
 -- Keep the top 2k singular vectors of W , so we get a new matrix $W_{2k} = U \Sigma_{2k} V$.
 -- Normalize W_k according to the momentum of the F-norm of W_{2k} .
 -- Truncate each coordinate of W_k to the top alpha percentile of all the coordinates, in absolute value.
 -- Apply momentum on W_k , and then update the network parameters using this momentum.
 -- Apply weight decay.

GPT-4:

```
def step(self):
    for group in self.param_groups:
        for p in group['params']:
            if p.grad is None:
                continue
            grad = p.grad.data
            # reshape grad to 2D
            grad_2d = grad.reshape(-1, group['d_dim'])
            # SVD decomposition
            U, S, V = torch.svd(grad_2d)
            # keep top k singular vectors
            G_k = U[:, :group['k']] @ torch.diag(S[:group['k']]) @ V[:, :group['k']].T
            # keep top 2k singular vectors
            G_2k = U[:, :2*group['k']] @ torch.diag(S[:2*group['k']]) @ V[:, :2*group['k']].T
            # normalize G_k
            G_k /= torch.norm(G_2k, p=2)
            # truncate G_k
            threshold = torch.quantile(torch.abs(G_k),
                                     group['alpha'])
            G_k[torch.abs(G_k) < threshold] = 0
            # apply momentum
            if 'momentum_buffer' not in group:
                group['momentum_buffer'] = torch.zeros_like(G_k)
            group['momentum_buffer'] = group['momentum_buffer'] *
            group['momentum_buffer'] + G_k
            # update parameters
            p.data.add_(-group['lr'], group['momentum_buffer'])
            # apply weight decay
            if group['weight_decay'] != 0:
                p.data.add_(-group['weight_decay'] * group['lr'],
                           p.data)
```

ChatGPT:

```
def step(self):
    for group in self.param_groups:
        for p in group['params']:
            if p.grad is None:
                continue
            grad = p.grad.data
            grad = grad.reshape(-1, grad.shape[-1]) # reshape to 2D
            # SVD decomposition
            u, s, v = torch.svd(grad)
            # Keep top k and 2k singular vectors
            k = group['k']
            u_k = u[:, :k]
            s_k = s[:k]
            v_k = v[:, :k]
            u_2k = u[:, :2*k]
            s_2k = s[:2*k]
            v_2k = v[:, :2*k]
            # Normalize according to momentum of F-norm of G_2k
            grad_k = u_k @ torch.diag(s_k) @ v_k.T()
            norm_2k = s_2k.norm()
            grad_k = grad_k * group['momentum'] / norm_2k
            # Truncate to top alpha percentile
            alpha = group['alpha']
            percentile = grad_k.abs().flatten().kthvalue(int(alpha *
            grad_k.numel()))
            grad_k[grad_k.abs() < percentile] = 0
            # Apply weight decay
            p.data.add_(-group['lr'], grad_k + group['weight_decay']
            * p.data)
```

Figure 3.4: GPT-4 vs ChatGPT at implementing a customized Pytorch optimizer module. We highlight the **astuteness** of GPT-4 vs the **mistake** by ChatGPT at ‘applying momentum’.

Video Generation

- Latent diffusion models use a low-dimensional latent space for efficiency
- **Key question:** how to generate multiple correlated frames?
- ‘Align your latents’ inserts temporal convolution / attention between each spatial convolution / attention
- ‘Preserve Your Own Correlation’ includes temporally correlated noise



SCALING UP

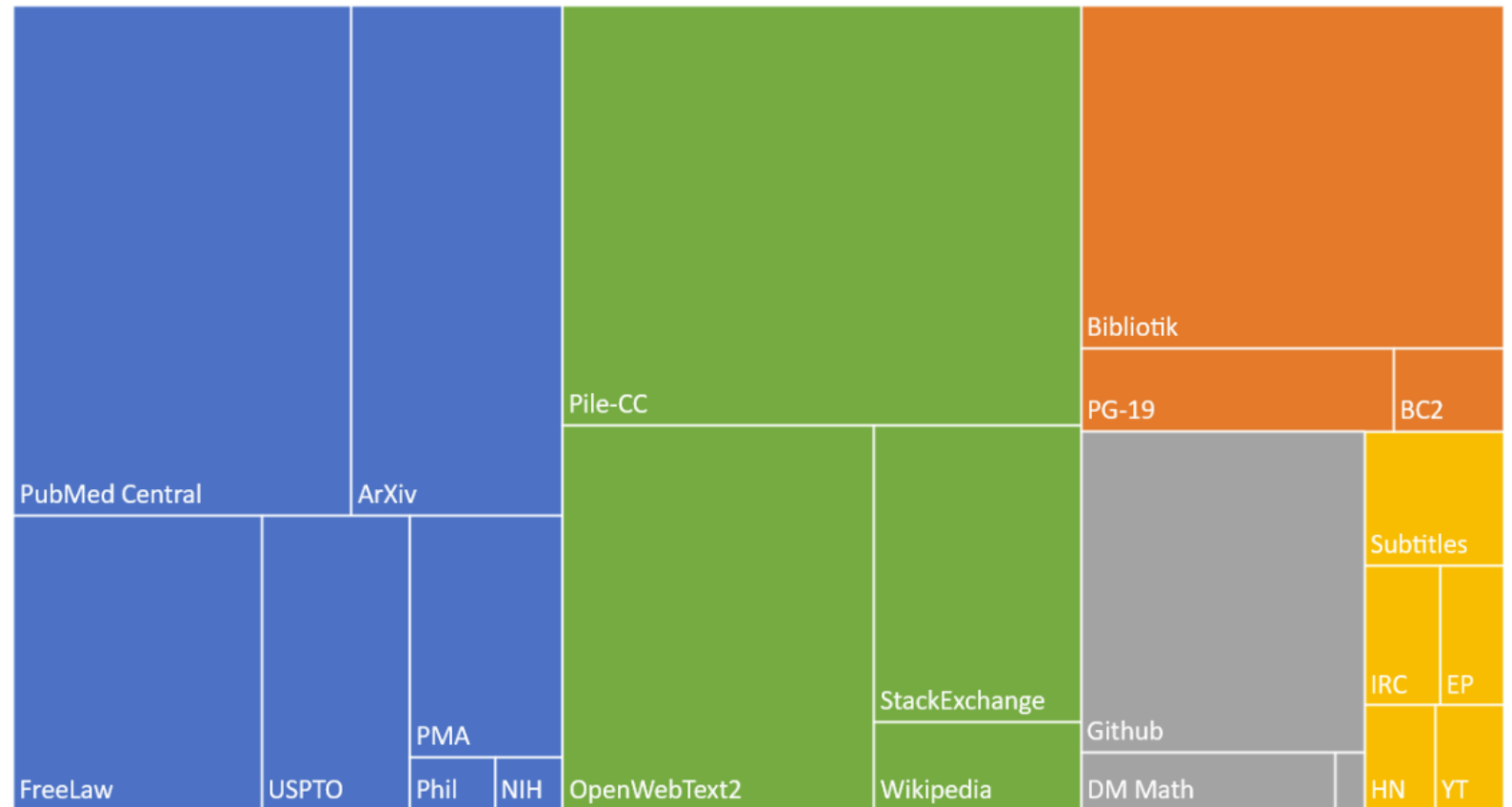
Training Data for LLMs

The Pile:

- An open source dataset for training language models
- Comprised of 22 smaller datasets
- Favors high quality text
- 825 Gb $\approx$ 1.2 trillion tokens

Composition of the Pile by Category

■ Academic ■ Internet ■ Prose ■ Dialogue ■ Misc



RLHF

- **InstructGPT** uses Reinforcement Learning with Human Feedback (RLHF) to **fine-tune a pre-trained GPT model**
- From the paper: “In human evaluations on our prompt distribution, outputs from the 1.3B parameter InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters.”

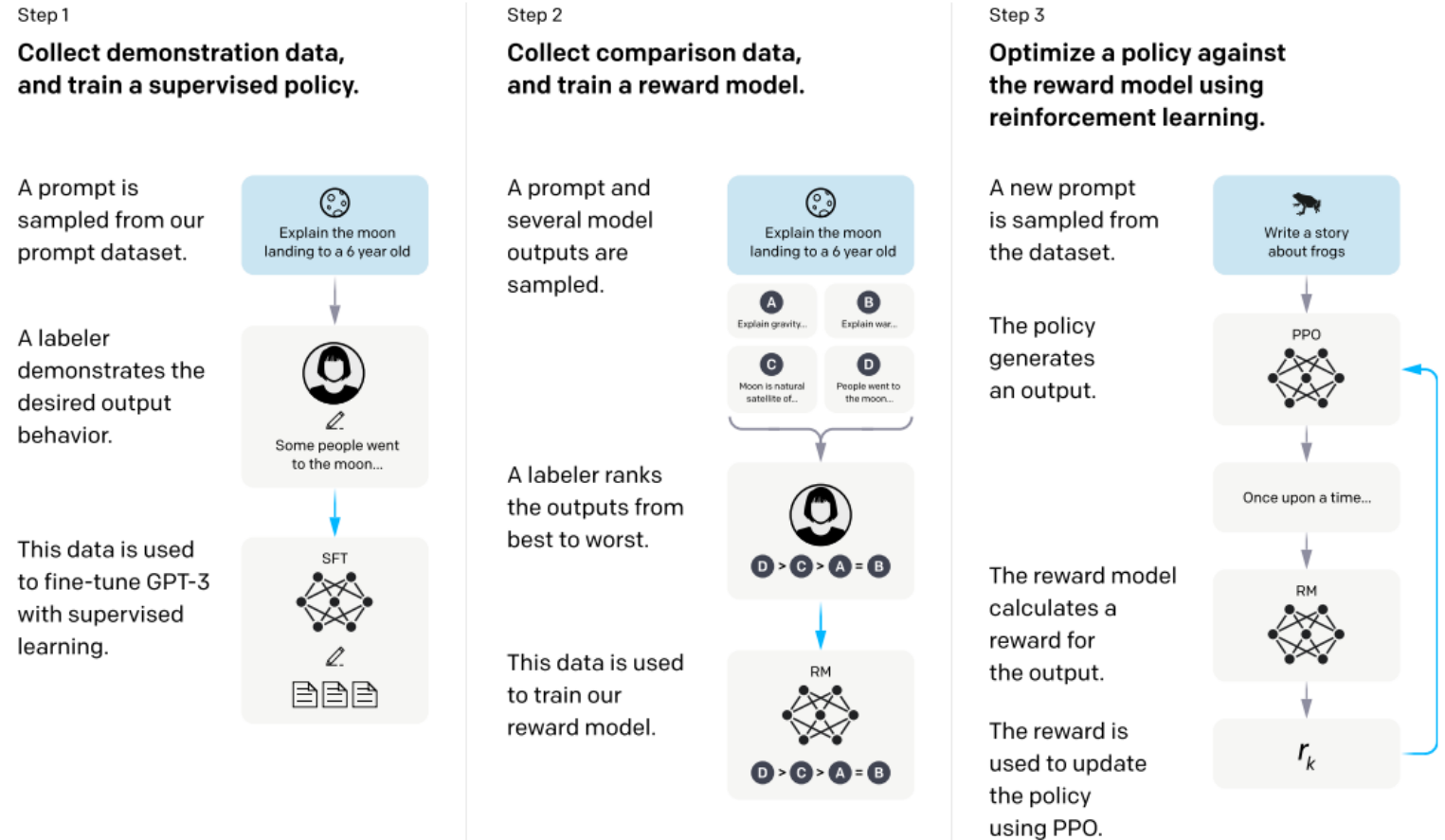


Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

Memory Usage of LLMs

How to store a large language model in memory?

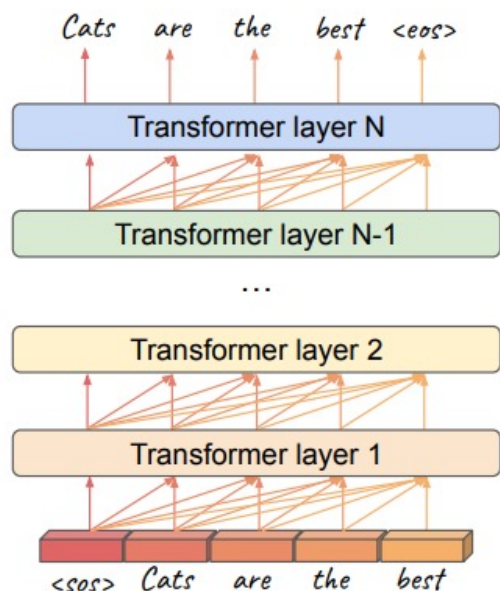
- **full precision:** 32-bit floats
- **half precision:** 16-bit floats
- Using half precision not only **reduces memory**, it also **speeds up** GPU computation
- “*Peak float16 matrix multiplication and convolution performance is 16x faster than peak float32 performance on A100 GPUs.*”

[from Pytorch docs](#)

Model	Megatron-LM	GPT-3
# parameters	8.3 billion	175 billion
full precision	30 Gb	651 Gb
half precision	15 Gb	325 Gb

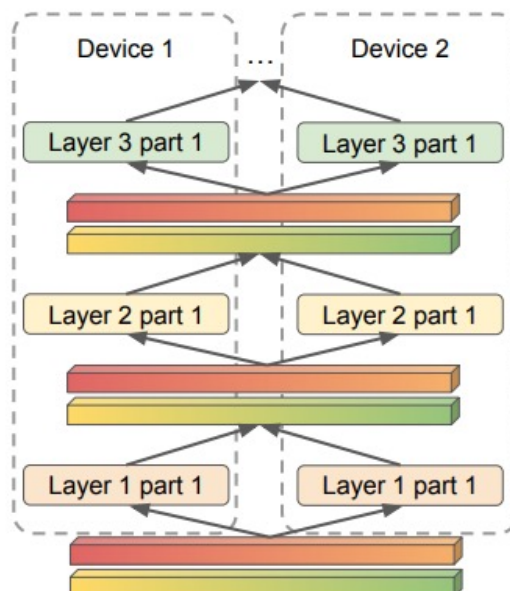
GPU / TPU	Max Memory
TPU v2	16 Gb
TPU v3/v4	32 Gb
Tesla V100 GPU	32 Gb
NVIDIA RTX A6000	48 Gb
Tesla A100 GPU	80 Gb

Distributed Training: Model Parallel



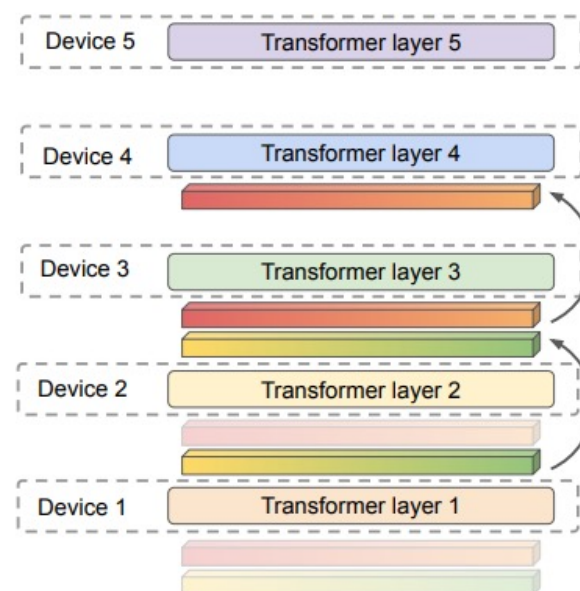
(a) Transformer-based LM

There are a variety of different options for how to distribute the model computation / parameters across multiple devices.



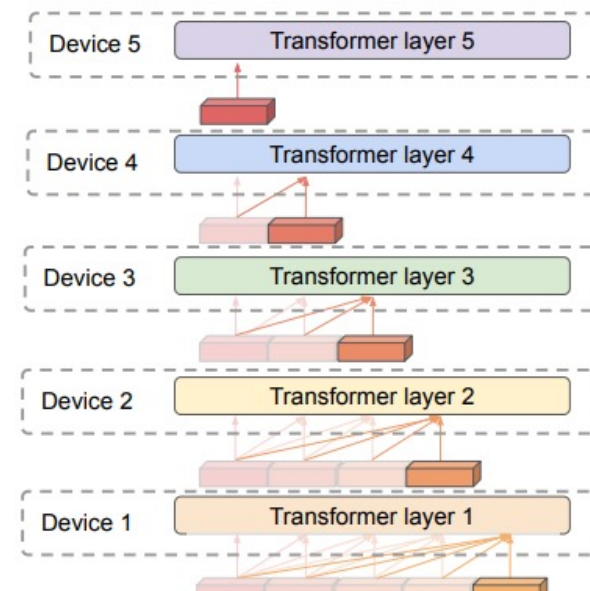
(b) Operation partitioning (Megatron-LM)

Matrix multiplication comprises most Transformer LM computation and can be divided along rows/columns of the respective matrices.



(c) Microbatch-based pipeline parallelism (GPipe)

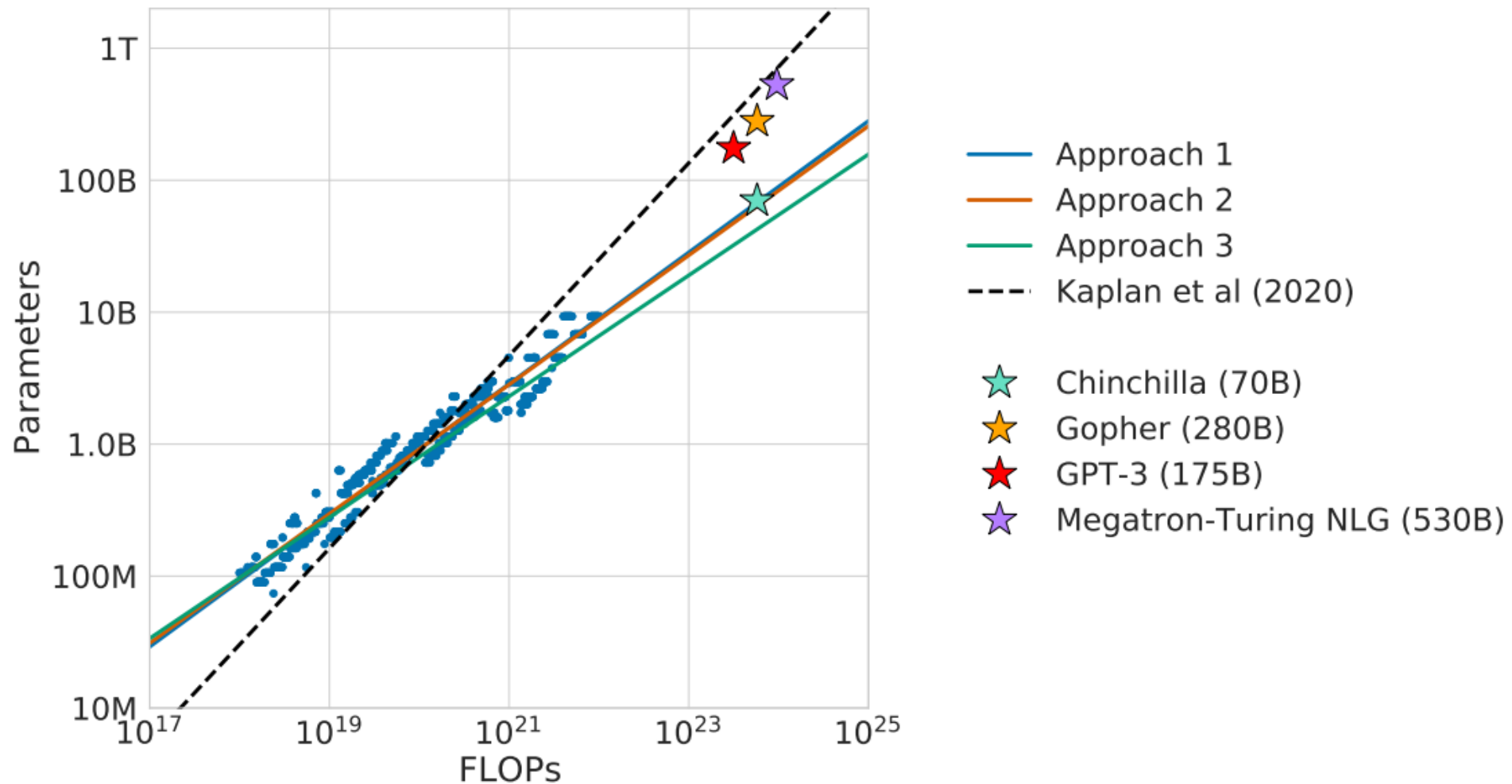
The most natural division is by layer: each device computes a subset of the layers, only that device stores the parameters and computation graph for those layers.



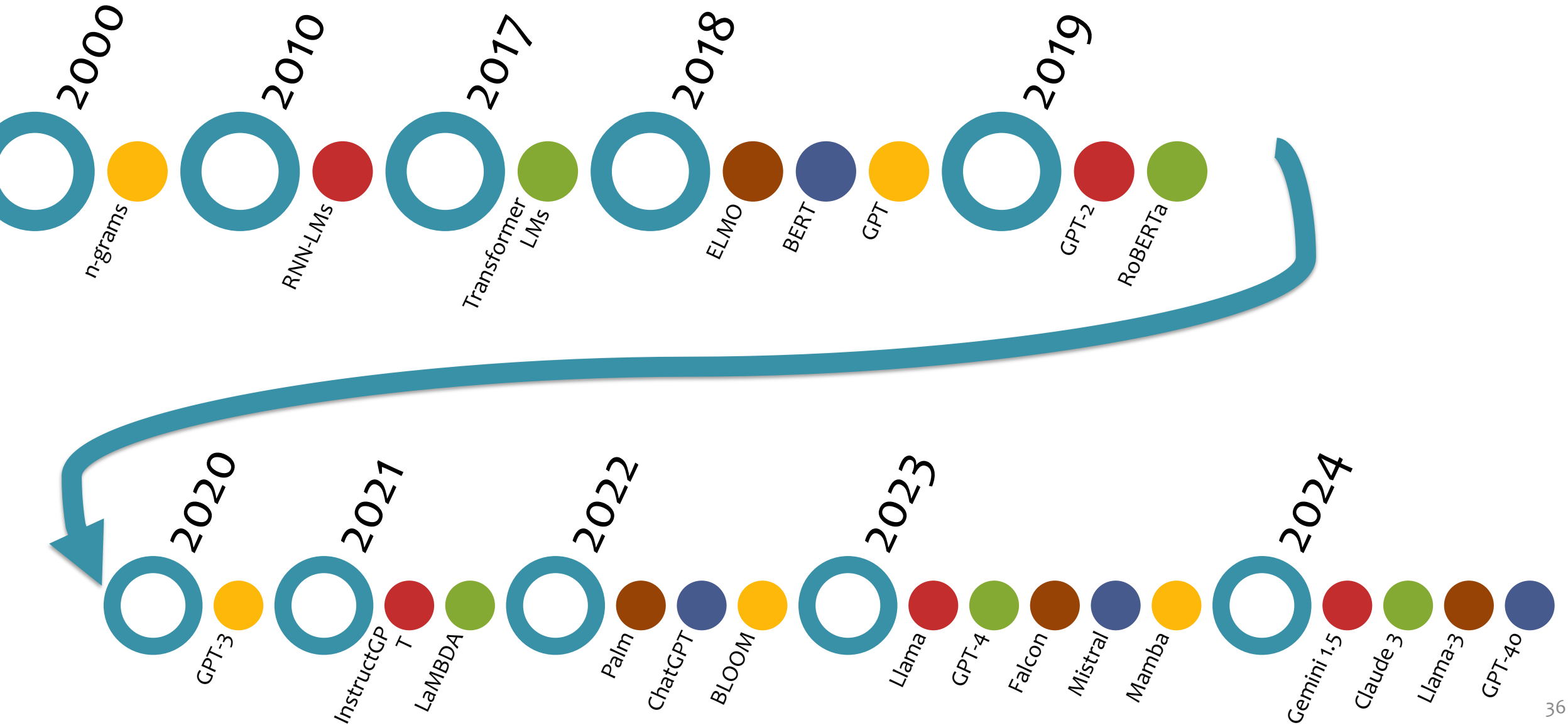
(d) Token-based pipeline parallelism (TeraPipe)

A more efficient solution is to divide computation by token *and* layer. This requires careful division of work and is specific to the Transformer LM.

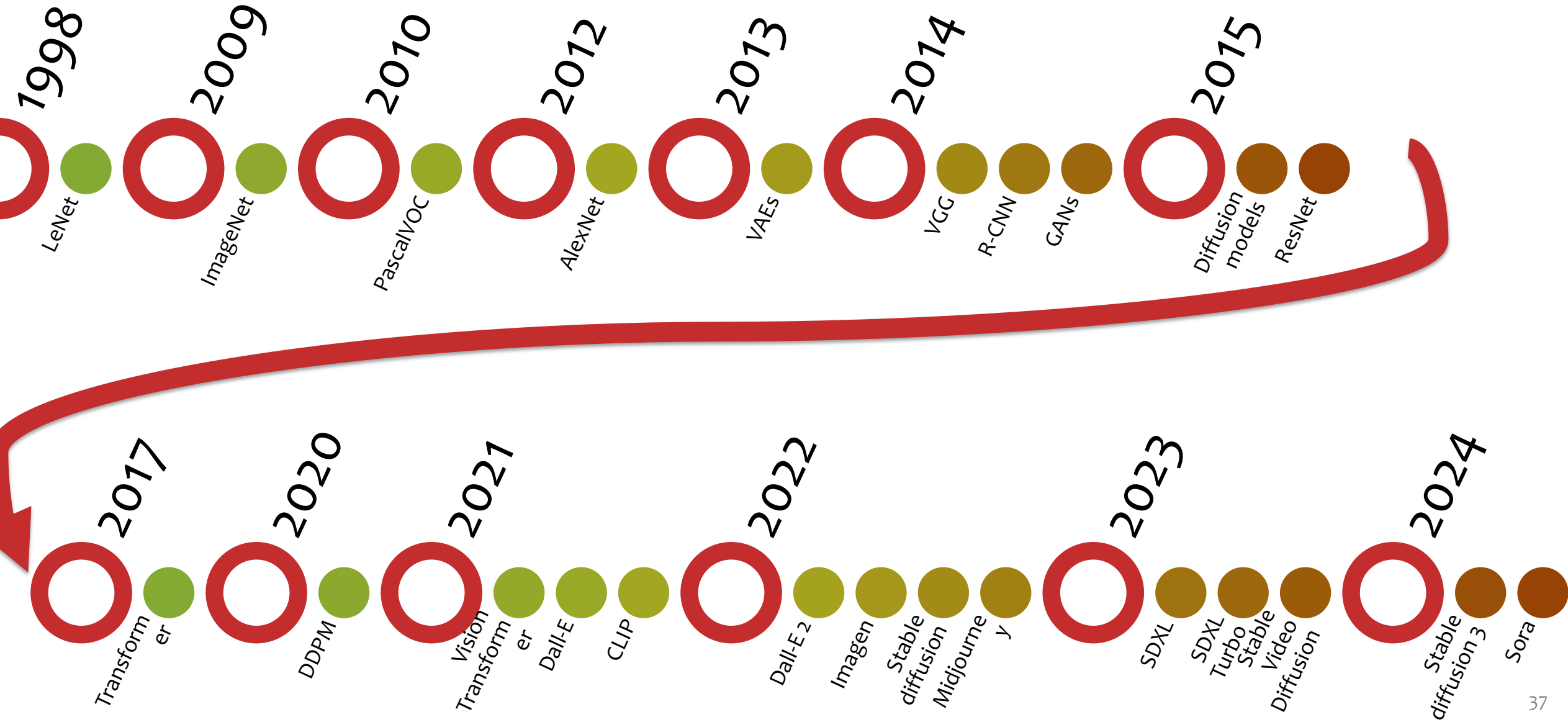
Cost to train



Timeline: Language Modeling



Timeline: Image Generation



GENERATIVE AI IS PROBABILISTIC MODELING

GenAI is Probabilistic Modeling

$$p(x_{t+1} \mid x_1, \dots, x_t)$$

**What if I want to model
EVERY possible
interaction?**

**...or at least the interactions of the
current variable with all those that came
before it...**

(RNN-LMs)

RNN Language Model

$$\text{RNN Language Model: } p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid f_{\theta}(w_{t-1}, \dots, w_1))$$

$$p(w_1, w_2, w_3, \dots, w_6) =$$

The						$p(w_1)$
The	bat					$p(w_2 \mid f_{\theta}(w_1))$
The	bat	made				$p(w_3 \mid f_{\theta}(w_2, w_1))$
The	bat	made	noise			$p(w_4 \mid f_{\theta}(w_3, w_2, w_1))$
The	bat	made	noise	at		$p(w_5 \mid f_{\theta}(w_4, w_3, w_2, w_1))$
The	bat	made	noise	at	night	$p(w_6 \mid f_{\theta}(w_5, w_4, w_3, w_2, w_1))$

Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t \mid f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector

**Why learn the inner-
workings of GenAI?**

(a metaphor)



Figure from https://en.wikipedia.org/wiki/Ford_Model_T



Figure from <https://daily.jstor.org/the-science-of-traffic/>



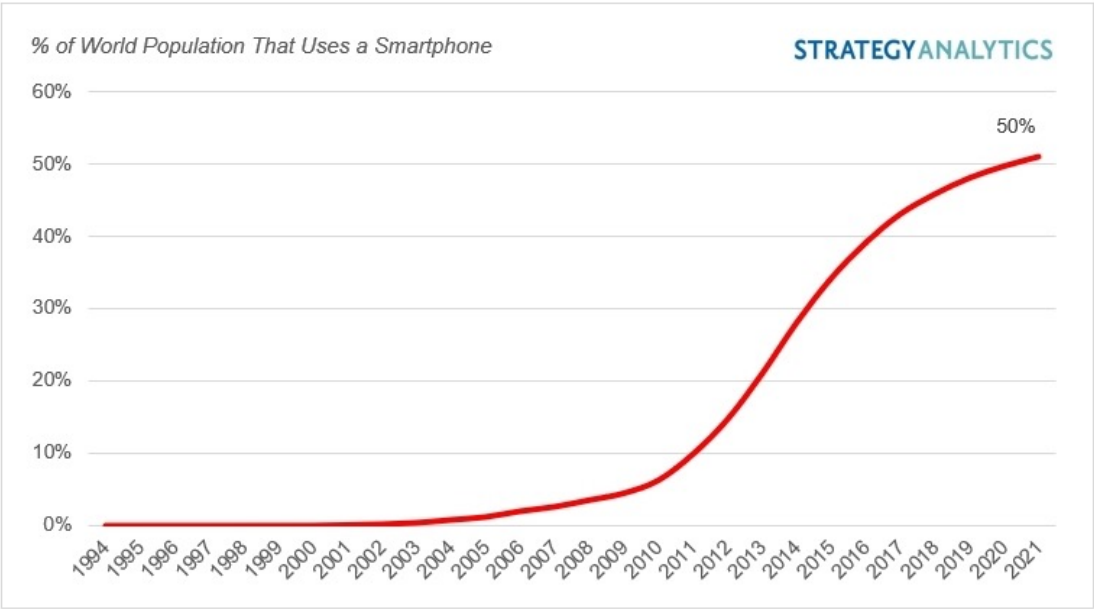
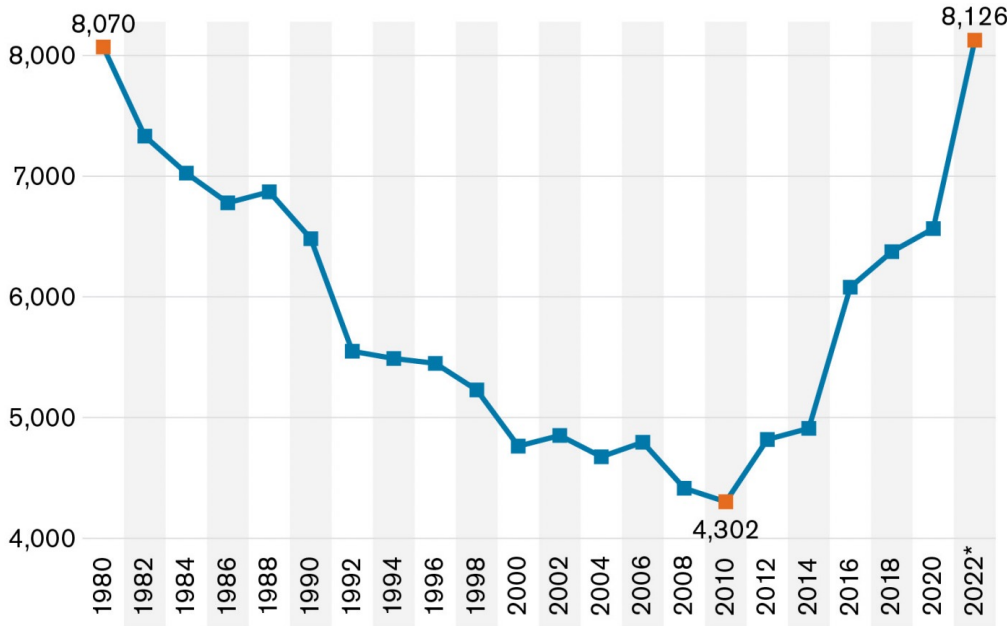


Figure 5 Number of Annual U.S. Pedestrian Fatalities, 1980-2022



Generative AI in the Wild

In-Class Exercise

1. Identify one setting where there is a big opportunity for Generative AI to have a positive impact on society / the world
2. Identify one setting where we all need to slow down and be cautious about Generative AI being applied

- In-Class Exercise**
1. Identify one setting where there is a big opportunity for Generative AI to have a positive impact on society / the world
 2. Identify one setting where we all need to slow down and be cautious about Generative AI being applied

Generative AI in the Wild

Big Opportunities	Caution Needed

THE OTHER KIND OF LEARNING

Claim A: The most important learning you do in this course will happen by struggling through challenging problems (and some programming) on the homework.

Claim B: Using a code assistant to solve a previously unsolved problem (unlike the heavily solved problems you'll be working on) is difficult because you must be able to...

1. read and understand many lines of generated code, and spot bugs amidst seemingly correct, well commented code
2. accurately articulate what the problem is and how you want to solve it (this is what our homework write ups do so well for your sake)
3. know and use best practices for software development (design patterns, unit testing, linting, packaging/reproducibility, version control)

Claim A: The most important learning you do in this course will happen by struggling through challenging problems (and some programming) on the homework.

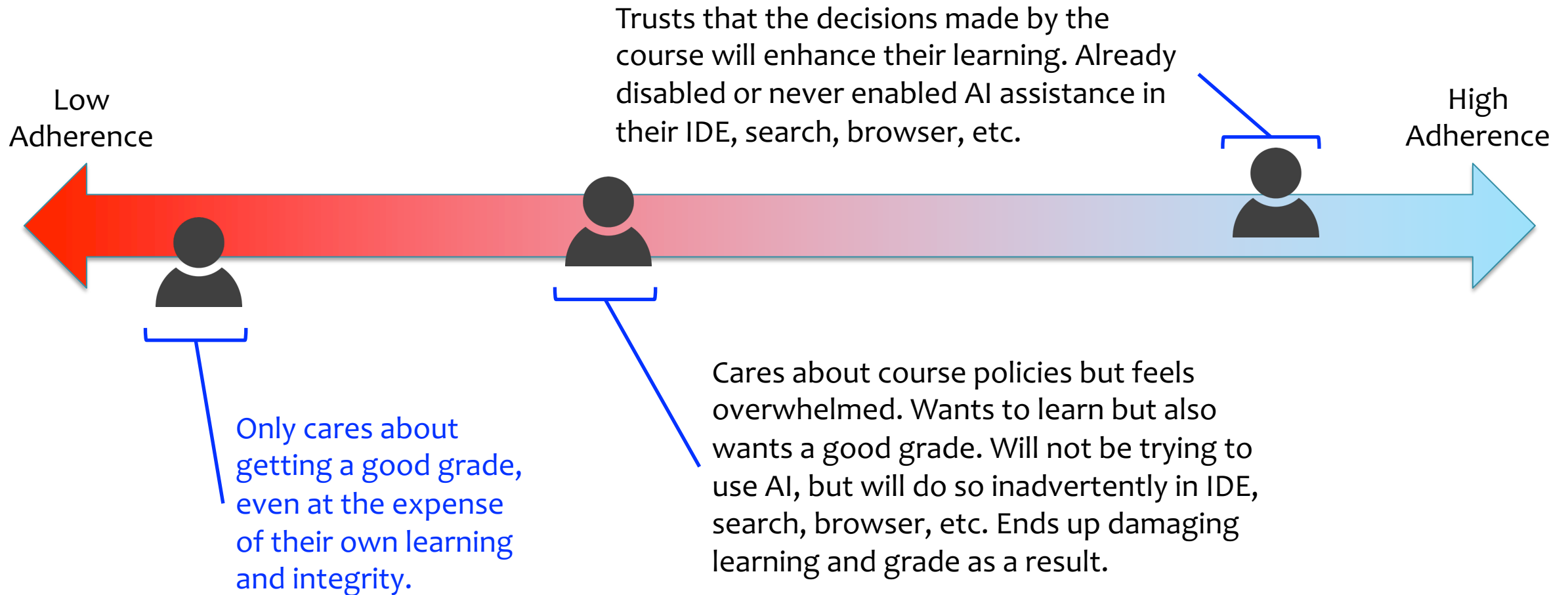
Building Good Habits

One of our goals in this course for many years has been to help students establish life-long learning habits that will aid you in this field for years to come

What about you:
where do you fall?

Building Good Habits

Imagine a course that disallows all use of AI, LLMs, code assistants.
Imagine placing students on a continuum of adherence to course policies.



Our Homework Policy

Each homework has two submission deadlines with *different rules*:

- **Slot A (Human work only):**
 - > Individual work with limited collaboration and no AI assistance of any kind.
 - > OH and Piazza active only during this submission period
- **Feedback (Human work by the course staff):**
 - > We grade your submission and inform you which questions you got incorrect.
- **Slot B (AI assisted work and full collaboration permitted):**
 - > Full collaboration and AI assistance are permitted.
 - > Due three days after you receive feedback.
 - > Then we grade only the questions you got incorrect.
- **Your HW Grade:**

$$s = 0.95 \times \left(\sum_{q \in HW} \max(s_{A,q}, s_{B,q}) \right) + 0.05 \times \mathbf{1}(s_A > 0.50)$$

Ungraded Homework

- Do some self reflection this week.
- Think about what kind of commitment you want to make to this course.
- Come talk to us.

SYLLABUS HIGHLIGHTS

Syllabus Highlights

The syllabus is located on the course webpage:

<http://423.mlcourse.org>

<http://623.mlcourse.org>

<http://723.mlcourse.org>



<https://www.cs.cmu.edu/~mgormley/courses/10423/>

The **course policies** are **required** reading.

Topics

- Generative models of text
 - RNN LMs / Autodiff
 - Transformer LMs
 - Pre-training, fine-tuning, evaluation, decoding
- Generative models of images
 - CNNs / Transformers for vision
 - GANs, Conditional GANs
 - VAEs and Diffusion models
- Applying and adapting foundation models
 - Reinforcement learning with human feedback (RLHF)
 - Parameter-efficient fine tuning
 - In-context learning
- Multimodal foundation models
 - Text-to-image generation
 - Aligning multimodal representations
 - Visual-language foundation models
- Scaling models
 - Efficient decoding strategies
 - Distributed training
 - Scaling laws and data
 - Mixture of experts
 - FlashAttention
- Advanced Topics
 - What can go wrong? Real-world Issues and Considerations: discrimination, bias, hallucination, copyright, adversarial attack, etc.
 - Long-context modeling
 - Reasoning models
 - State space models
 - Code generation / Autonomous Agents
 - Audio understanding and synthesis
 - Video synthesis
 - Interactive world models

Syllabus Highlights

- **Grading:** 40% homework, 10% quizzes, 20% exam, 25% project, 5% participation
- **Exam:** in-class exam, Mon, Mar. 31
- **Homework:** 5 assignments
 - 6 grace days for homework assignments
 - Late submissions: 75% day 1, 50% day 2, 25% day 3
 - No submissions accepted after 4 days w/o extension
 - Extension requests: only for emergency situations, see syllabus
- **Recitations:** Fridays, same time/place as lecture (optional, interactive sessions)
- **Readings:** required, online PDFs, recommended for after lecture
- **Technologies:**
 - Piazza (discussion),
 - Gradescope (homework),
 - Google Forms (polls),
 - Zoom (livestream),
 - Panopto (video recordings)
- **Academic Integrity:**
 - Collaboration encouraged, but must be documented
 - Solutions must always be written independently
 - No use of found code / past work
 - No use of AI tools to complete HW
 - (Policies differ from 10-301/10-601)
- **Office Hours:** posted on Google Calendar on “Office Hours” page

Lectures

- You should ask lots of questions
 - Interrupting (by raising a hand) to ask your question is strongly encouraged
 - Asking questions later (or in real time) on Piazza is also great
- When I ask a question...
 - I want you to answer
 - Even if you don't answer, think it through as though I'm about to call on you
- Interaction improves learning (both in-class and at my office hours)

Prerequisites

What they are:

Introductory machine learning.
(i.e. 10-301, 10-315, 10-601, 10-701)

If you instead took an introduction
to deep learning course, that is
also fine
(i.e. 11-485/11-685/11-785)

What is not required:

- Deep learning
- PyTorch



Depending on which prerequisite course you took and in which semester you took it, you may or may not have been exposed to deep learning and/or PyTorch. Either way is fine.

Homework

There will be 5 homework assignments during the semester. The assignments will consist of both conceptual and programming problems.

	Main Topic	Implementation	Application Area	Type
HW0	PyTorch Primer	image classifier + Text classifier	vision + language	written + programming
HW1	Large Language Models	TransformerLM with GQA and RoPE	text gen	written + programming
HW2	Image Generation	Diffusion model	image gen	written + programming
HW3	Adapters for LLMs	GPT-2 + LoRA	instruction fine-tuning	written + programming
HW4	Multimodal Foundation Models	text-to-image editing model	vision + language	written + programming
HW623	(10-623 only)	read / analyze a recent research paper	genAI	video presentation

Project

- Goals:
 - Explore a generative modeling technique of your choosing
 - Deeper understanding of methods in real-world application
 - Report back to the class during a *poster session* to be held **sometime over finals period**
 - Work in teams of 3 students



Prompt to ChatGPT-4o: *Create an image of three Scottish terriers in traditional Scottish outfits working collaboratively on a project for a generative AI course*

Textbooks

... do not exist for this course.

Instead, we will be directing
your reading time to current
research papers.

Where can I find...?

[Home](#)
[FAQ](#)
[Syllabus](#)
[People](#)
[Schedule](#)
[Office Hours](#)
[Coursework](#)
[Links](#)

10-423 + 10-623,
School of Computer Science
Carnegie Mellon University

Generative AI

[Jump to Latest \(Lecture 1\)](#) [Open Latest Poll](#)

Important Notes

This schedule is **tentative** and subject to change. Please check back often.

Tentative Schedule

Date	Lecture	Readings	Announcements
Generative models of text			
Wed, 17-Jan	Lecture 1 : RNN LMs / Autodiff		HW0 out
Fri, 19-Jan	Recitation: HW0		
Mon, 22-Jan	Lecture 2 : Transformer LMs		
Wed, 24-Jan	Lecture 3 : Pre-training, fine-tuning, evaluation, decoding		HW0 due HW1 out (L1-L3)
Fri, 26-Jan	Recitation: HW1		

Where can I find...?

[Home](#)[FAQ](#)[Syllabus](#)[People](#)[Schedule](#)[Office Hours](#)[Coursework](#)[Links](#)

Generative AI

10-423 + 10-623,
School of Computer Science
Carnegie Mellon University

10-423/10-623 Office Hours

Today ◀ ▶ Jan 14 – 20, 2024 ▼

Week Month Agenda

	Sun 1/14	Mon 1/15	Tue 1/16	Wed 1/17	Thu 1/18	Fri 1/19	Sat 1/20
11am							
12pm							
1pm							
2pm							
3pm							
4pm							
5pm							
6pm							
7pm							
8pm							

3:30p – 4:50p
10-423/10-623
GenAI
GHC 4401

3:30p – 4:50p
10-423/10-623
GenAI
GHC 4401

Where can I find...?

[Home](#)
[FAQ](#)
[Syllabus](#)
[People](#)
[Schedule](#)
[Office Hours](#)
[Coursework](#)
[Links](#)

Generative AI

10-423 + 10-623, !
School of Computer Science
Carnegie Mellon University

Assignments

There will be 5 homework assignments (and a special extra assignment for 10-623 only). The assignments will consist of both theoretical and programming problems. Homework assignments will be released via a Piazza announcement explaining where to find the handout, LaTeX template, etc.

- Homework 0: PyTorch Primer
- Homework 1: Large Language Models
- Homework 2: Image Generation
- Homework 3: Adapters for LLMs
- Homework 4: Multimodal Foundation Models
- Homework 623: (10-623 only)

Tentative release dates and due dates are listed on the [Schedule](#) page.

Quizzes

There will be 5 quizzes.

- Quiz 1 (Lectures 1 - 3)
- Quiz 2 (Lectures 4 - 7)
- Quiz 3 (Lectures 8 - 11)
- Quiz 4 (Lectures 12 - 15)
- Quiz 5 (Lectures 16 - 20)

Reminders

- **Homework 0: PyTorch + Weights & Biases**
 - Out: Wed, Aug 27
 - Due: Mon, Sep 8 at 11:59pm
 - Two parts:
 1. written part to Gradescope
 2. programming part to Gradescope
 - **unique policy for this assignment: we will grant (essentially) any and all extension requests**

Learning Objectives

You should be able to...

1. Differentiate between different mechanisms of learning such as parameter tuning and in-context learning.
2. Implement the foundational models underlying modern approaches to generative modeling, such as transformers and diffusion models.
3. Apply existing models to real-world generation problems for text, code, images, audio, and video.
4. Employ techniques for adapting foundation models to tasks such as fine-tuning, adapters, and in-context learning.
5. Enable methods for generative modeling to scale-up to large datasets of text, code, or images.
6. Use existing generative models to solve real-world discriminative problems and for other everyday use cases.
7. Analyze the theoretical properties of foundation models at scale.
8. Identify potential pitfalls of generative modeling for different modalities.
9. Describe societal impacts of large-scale generative AI systems.

Q&A

MODULE-BASED AUTOMATIC DIFFERENTIATION

Backpropagation

Automatic Differentiation – Reverse Mode (aka. Backpropagation)

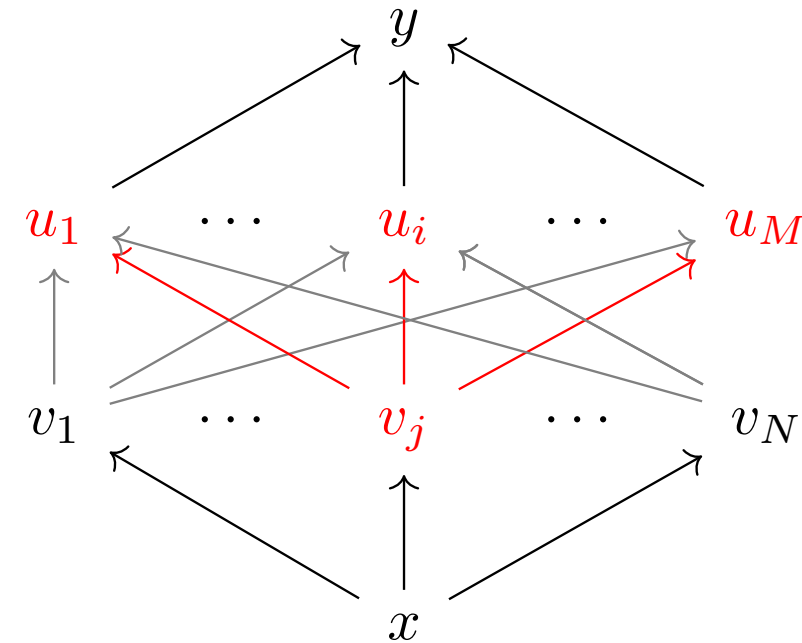
Forward Computation

1. Write an **algorithm** for evaluating the function $y = f(\mathbf{x})$. The algorithm defines a **directed acyclic graph**, where each variable is a node (i.e. the “**computation graph**”)
2. Visit each node in **topological order**.
For variable u_i with inputs $v_1, \dots, v_N$
 - a. Compute $u_i = g_i(v_1, \dots, v_N)$
 - b. Store the result at the node

Backward Computation (Version A)

1. **Initialize** $dy/dy = 1$.
2. Visit each node v_j in **reverse topological order**.
Let $u_1, \dots, u_M$ denote all the nodes with v_j as an input
Assuming that $y = h(\mathbf{u}) = h(u_1, \dots, u_M)$
and $\mathbf{u} = \mathbf{g}(\mathbf{v})$ or equivalently $u_i = g_i(v_1, \dots, v_j, \dots, v_N)$ for all i
 - a. We already know dy/du_i for all i
 - b. Compute dy/dv_j as below (Choice of algorithm ensures computing (du_i/dv_j) is easy)

$$\frac{dy}{dv_j} = \sum_{i=1}^M \frac{dy}{du_i} \frac{du_i}{dv_j}$$



Return partial derivatives dy/du_i for all variables

Backpropagation

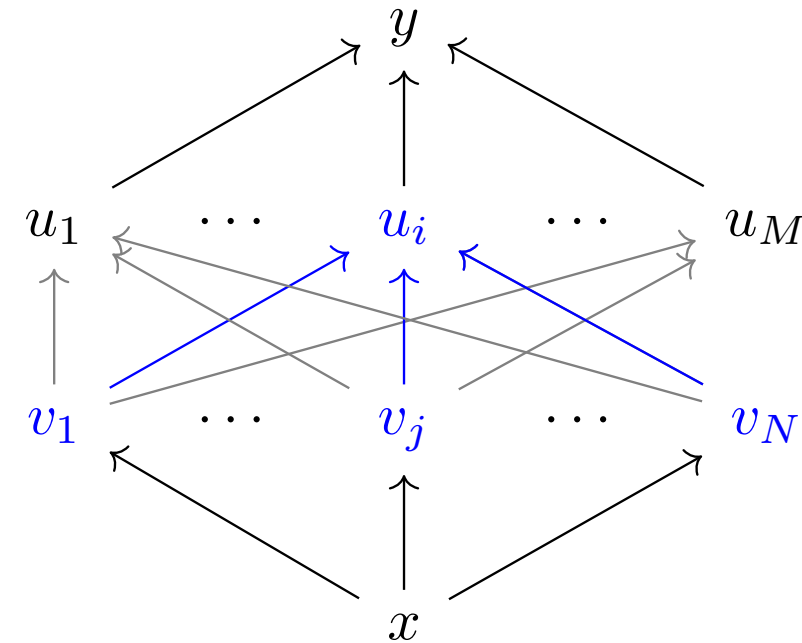
Automatic Differentiation – Reverse Mode (aka. Backpropagation)

Forward Computation

1. Write an **algorithm** for evaluating the function $y = f(\mathbf{x})$. The algorithm defines a **directed acyclic graph**, where each variable is a node (i.e. the “**computation graph**”)
2. Visit each node in **topological order**.
For variable u_i with inputs $v_1, \dots, v_N$
 - a. Compute $u_i = g_i(v_1, \dots, v_N)$
 - b. Store the result at the node

Backward Computation (Version B)

1. **Initialize** all partial derivatives dy/du_i to 0 and $dy/dy = 1$.
2. Visit each node in **reverse topological order**.
For variable $u_i = g_i(v_1, \dots, v_N)$
 - a. We already know dy/du_i
 - b. Increment dy/dv_j by $(dy/du_i)(du_i/dv_j)$
(Choice of algorithm ensures computing (du_i/dv_j) is easy)



Return partial derivatives dy/du_i for all variables

Backpropagation: Procedural Method

Algorithm 1 Forward Computation

```
1: procedure NNFORWARD(Training example  $(\mathbf{x}, \mathbf{y})$ , Params  $\alpha, \beta$ )
2:    $\mathbf{a} = \alpha \mathbf{x}$ 
3:    $\mathbf{z} = \sigma(\mathbf{a})$ 
4:    $\mathbf{b} = \beta \mathbf{z}$ 
5:    $\hat{\mathbf{y}} = \text{softmax}(\mathbf{b})$ 
6:    $J = -\mathbf{y}^T \log \hat{\mathbf{y}}$ 
7:    $\mathbf{o} = \text{object}(\mathbf{x}, \mathbf{a}, \mathbf{z}, \mathbf{b}, \hat{\mathbf{y}}, J)$ 
8:   return intermediate quantities  $\mathbf{o}$ 
```

Algorithm 2 Backpropagation

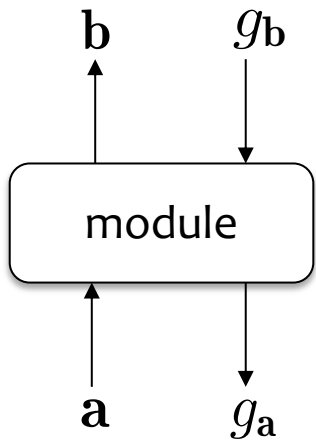
```
1: procedure NNBACKWARD(Training example  $(\mathbf{x}, \mathbf{y})$ , Params  $\alpha, \beta$ ,  
  Intermediates  $\mathbf{o}$ )
2:   Place intermediate quantities  $\mathbf{x}, \mathbf{a}, \mathbf{z}, \mathbf{b}, \hat{\mathbf{y}}, J$  in  $\mathbf{o}$  in scope
3:    $\mathbf{g}_{\hat{\mathbf{y}}} = -\mathbf{y} \div \hat{\mathbf{y}}$ 
4:    $\mathbf{g}_{\mathbf{b}} = \mathbf{g}_{\hat{\mathbf{y}}}^T (\text{diag}(\hat{\mathbf{y}}) - \hat{\mathbf{y}} \hat{\mathbf{y}}^T)$ 
5:    $\mathbf{g}_{\beta} = \mathbf{g}_{\mathbf{b}}^T \mathbf{z}^T$ 
6:    $\mathbf{g}_{\mathbf{z}} = \beta^T \mathbf{g}_{\mathbf{b}}^T$ 
7:    $\mathbf{g}_{\mathbf{a}} = \mathbf{g}_{\mathbf{z}} \odot \mathbf{z} \odot (1 - \mathbf{z})$ 
8:    $\mathbf{g}_{\alpha} = \mathbf{g}_{\mathbf{a}} \mathbf{x}^T$ 
9:   return parameter gradients  $\mathbf{g}_{\alpha}, \mathbf{g}_{\beta}$ 
```

Drawbacks of Procedural Method

1. Hard to reuse / adapt for other models
2. (Possibly) harder to make individual steps more efficient
3. Hard to find source of error if finite-difference check reports an error (since it tells you only that there is an error somewhere in those 17 lines of code)

Module-based AutoDiff

- **Key Idea:**
 - componentize the computation of the neural-network into layers
 - each layer consolidates multiple **real-valued nodes** in the computation graph (a subset of them) into one **vector-valued node** (aka. a **module**)
- Each **module** is capable of two actions:
 1. Forward computation of output $\mathbf{b} = [b_1, \dots, b_B]$ given input $\mathbf{a} = [a_1, \dots, a_A]$ via some differentiable function f . That is $\mathbf{b} = f(\mathbf{a})$.
 2. Backward computation of the gradient of the input $\mathbf{g}_a = \nabla_{\mathbf{a}} J = [\frac{\partial J}{\partial a_1}, \dots, \frac{\partial J}{\partial a_A}]$ given the gradient of output $\mathbf{g}_b = \nabla_{\mathbf{b}} J = [\frac{\partial J}{\partial b_1}, \dots, \frac{\partial J}{\partial b_B}]$, where J is the final real-valued output of the entire computation graph. This is done via the chain rule $\frac{\partial J}{\partial a_i} = \sum_{j=1}^B \frac{\partial J}{\partial b_j} \frac{db_j}{da_i}$ for all $i \in \{1, \dots, A\}$.



Module-based AutoDiff

Dimensions: input $\mathbf{a} \in \mathbb{R}^A$, output $\mathbf{b} \in \mathbb{R}^B$, gradient of output $\mathbf{g}_\mathbf{a} \triangleq \nabla_\mathbf{a} J \in \mathbb{R}^A$, and gradient of input $\mathbf{g}_\mathbf{b} \triangleq \nabla_\mathbf{b} J \in \mathbb{R}^B$.

Sigmoid Module The sigmoid layer has only one input vector $\mathbf{a}$. Below σ is the sigmoid applied element-wise, and $\odot$ is element-wise multiplication s.t. $\mathbf{u} \odot \mathbf{v} = [u_1 v_1, \dots, u_M v_M]$.

```
1: procedure SIGMOIDFORWARD( $\mathbf{a}$ )
2:    $\mathbf{b} = \sigma(\mathbf{a})$ 
3:   return  $\mathbf{b}$ 
4: procedure SIGMOIDBACKWARD( $\mathbf{a}, \mathbf{b}, \mathbf{g}_\mathbf{b}$ )
5:    $\mathbf{g}_\mathbf{a} = \mathbf{g}_\mathbf{b} \odot \mathbf{b} \odot (1 - \mathbf{b})$ 
6:   return  $\mathbf{g}_\mathbf{a}$ 
```

Softmax Module The softmax layer has only one input vector $\mathbf{a}$. For any vector $\mathbf{v} \in \mathbb{R}^D$, we have that $\text{diag}(\mathbf{v})$ returns a $D \times D$ diagonal matrix whose diagonal entries are $v_1, v_2, \dots, v_D$ and whose non-diagonal entries are zero.

```
1: procedure SOFTMAXFORWARD( $\mathbf{a}$ )
2:    $\mathbf{b} = \text{softmax}(\mathbf{a})$ 
3:   return  $\mathbf{b}$ 
4: procedure SOFTMAXBACKWARD( $\mathbf{a}, \mathbf{b}, \mathbf{g}_\mathbf{b}$ )
5:    $\mathbf{g}_\mathbf{a} = \mathbf{g}_\mathbf{b}^T (\text{diag}(\mathbf{b}) - \mathbf{b} \mathbf{b}^T)$ 
6:   return  $\mathbf{g}_\mathbf{a}$ 
```

Linear Module The linear layer has two inputs: a vector $\mathbf{a}$ and parameters $\omega \in \mathbb{R}^{B \times A}$. The output $\mathbf{b}$ is not used by LINEARBACKWARD, but we pass it in for consistency of form.

```
1: procedure LINEARFORWARD( $\mathbf{a}, \omega$ )
2:    $\mathbf{b} = \omega \mathbf{a}$ 
3:   return  $\mathbf{b}$ 
4: procedure LINEARBACKWARD( $\mathbf{a}, \omega, \mathbf{b}, \mathbf{g}_\mathbf{b}$ )
5:    $\mathbf{g}_\omega = \mathbf{g}_\mathbf{b} \mathbf{a}^T$ 
6:    $\mathbf{g}_\mathbf{a} = \omega^T \mathbf{g}_\mathbf{b}$ 
7:   return  $\mathbf{g}_\omega, \mathbf{g}_\mathbf{a}$ 
```

Cross-Entropy Module The cross-entropy layer has two inputs: a gold one-hot vector $\mathbf{a}$ and a predicted probability distribution $\hat{\mathbf{a}}$. Its output $b \in \mathbb{R}$ is a scalar. Below $\div$ is element-wise division. The output b is not used by CROSSENTROPYBACKWARD, but we pass it in for consistency of form.

```
1: procedure CROSSENTROPYFORWARD( $\mathbf{a}, \hat{\mathbf{a}}$ )
2:    $b = -\mathbf{a}^T \log \hat{\mathbf{a}}$ 
3:   return  $\mathbf{b}$ 
4: procedure CROSSENTROPYBACKWARD( $\mathbf{a}, \hat{\mathbf{a}}, b, \mathbf{g}_b$ )
5:    $\mathbf{g}_{\hat{\mathbf{a}}} = -\mathbf{g}_b (\mathbf{a} \div \hat{\mathbf{a}})$ 
6:   return  $\mathbf{g}_\mathbf{a}$ 
```

Module-based AutoDiff

Algorithm 1 Forward Computation

```
1: procedure NNFORWARD(Training example (x, y), Parameters  $\alpha$ ,  
    $\beta$ )  
2:   a = LINEARFORWARD(x,  $\alpha$ )  
3:   z = SIGMOIDFORWARD(a)  
4:   b = LINEARFORWARD(z,  $\beta$ )  
5:    $\hat{\mathbf{y}}$  = SOFTMAXFORWARD(b)  
6:    $J$  = CROSSENTROPYFORWARD(y,  $\hat{\mathbf{y}}$ )  
7:   o = object(x, a, z, b,  $\hat{\mathbf{y}}$ ,  $J$ )  
8:   return intermediate quantities o
```

Algorithm 2 Backpropagation

```
1: procedure NNBACKWARD(Training example (x, y), Parameters  
    $\alpha$ ,  $\beta$ , Intermediates o)  
2:   Place intermediate quantities x, a, z, b,  $\hat{\mathbf{y}}$ ,  $J$  in o in scope  
3:    $g_J = \frac{dJ}{dJ} = 1$  ▷ Base case  
4:    $\mathbf{g}_{\hat{\mathbf{y}}} = \text{CROSSENTROPYBACKWARD}(\mathbf{y}, \hat{\mathbf{y}}, J, g_J)$   
5:    $\mathbf{g}_{\mathbf{b}} = \text{SOFTMAXBACKWARD}(\mathbf{b}, \hat{\mathbf{y}}, \mathbf{g}_{\hat{\mathbf{y}}})$   
6:    $\mathbf{g}_{\beta}, \mathbf{g}_{\mathbf{z}} = \text{LINEARBACKWARD}(\mathbf{z}, \mathbf{b}, \mathbf{g}_{\mathbf{b}})$   
7:    $\mathbf{g}_{\mathbf{a}} = \text{SIGMOIDBACKWARD}(\mathbf{a}, \mathbf{z}, \mathbf{g}_{\mathbf{z}})$   
8:    $\mathbf{g}_{\alpha}, \mathbf{g}_{\mathbf{x}} = \text{LINEARBACKWARD}(\mathbf{x}, \mathbf{a}, \mathbf{g}_{\mathbf{a}})$  ▷ We discard  $\mathbf{g}_{\mathbf{x}}$   
9:   return parameter gradients  $\mathbf{g}_{\alpha}, \mathbf{g}_{\beta}$ 
```

Advantages of Module-based AutoDiff

1. Easy to reuse / adapt for other models
2. Encapsulated layers are easier to optimize (e.g. implement in C++ or CUDA)
3. Easier to find bugs because we can run a finite-difference check on each layer separately

Module-based AutoDiff (oOP Version)

Object-Oriented Implementation:

- Let each module be an **object**
- Then allow the **control flow** dictate the creation of the **computation graph**
- No longer need to implement NNBackward($\cdot$), just follow the computation graph in **reverse topological order**

```
1 class Sigmoid(Module)
2     method forward(a)
3          $b = \sigma(a)$ 
4         return b
5     method backward(a, b, gb)
6          $g_a = g_b \odot b \odot (1 - b)$ 
7         return ga
```

```
1 class Softmax(Module)
2     method forward(a)
3          $b = \text{softmax}(a)$ 
4         return b
5     method backward(a, b, gb)
6          $g_a = g_b^T (\text{diag}(b) - bb^T)$ 
7         return ga
```

```
1 class Linear(Module)
2     method forward(a,  $\omega$ )
3          $b = \omega a$ 
4         return b
5     method backward(a,  $\omega$ , b, gb)
6          $g_\omega = g_b a^T$ 
7          $g_a = \omega^T g_b$ 
8         return g $_\omega$ , ga
```

```
1 class CrossEntropy(Module)
2     method forward(a,  $\hat{a}$ )
3          $b = -a^T \log \hat{a}$ 
4         return b
5     method backward(a,  $\hat{a}$ , b, gb)
6          $g_{\hat{a}} = -g_b (a \div \hat{a})$ 
7         return ga
```

Module-based AutoDiff (oOP Version)

```
1 class NeuralNetwork(Module):
2
3     method init()
4         lin1_layer = Linear()
5         sig_layer = Sigmoid()
6         lin2_layer = Linear()
7         soft_layer = Softmax()
8         ce_layer = CrossEntropy()
9
10    method forward(Tensor x, Tensor y, Tensor  $\alpha$ , Tensor  $\beta$ )
11        a = lin1_layer.apply_fwd(x,  $\alpha$ )
12        z = sig_layer.apply_fwd(a)
13        b = lin2_layer.apply_fwd(z,  $\beta$ )
14         $\hat{y}$  = soft_layer.apply_fwd(b)
15        J = ce_layer.apply_fwd(y,  $\hat{y}$ )
16        return J.out_tensor
17
18    method backward(Tensor x, Tensor y, Tensor  $\alpha$ , Tensor  $\beta$ )
19        tape_bwd()
20        return lin1_layer.in_gradients[1], lin2_layer.in_gradients[1]
```

```
1 global tape = stack()
2
3 class Module:
4
5     method init()
6         out_tensor = null
7         out_gradient = Tensor(1)
8
9     method apply_fwd(List in_modules)
10        in_tensors = [x.out_tensor for x in in_modules]
11        out_tensor = forward(in_tensors)
12        tape.push(self)
13        return self
14
15    method apply_bwd():
16        in_gradients = backward(in_tensors, out_tensor, out_gradient)
17        for i in 1, ..., len(in_modules):
18            in_modules[i].out_gradient += in_gradients[i]
19        return self
20
21    function tape_bwd():
22        while len(tape) > 0
23            m = tape.pop()
24            m.apply_bwd()
```


Module-based AutoDiff (oOP Version)

```
1 class NeuralNetwork(Module):
2
3     method init()
4         lin1_layer = Linear()
5         sig_layer = Sigmoid()
6         lin2_layer = Linear()
7         soft_layer = Softmax()
8         ce_layer = CrossEntropy()
9
10    method forward(Tensor  $\mathbf{x}$ , Tensor  $\mathbf{y}$ , Tensor  $\boldsymbol{\alpha}$ , Tensor  $\boldsymbol{\beta}$ )
11         $\mathbf{a}$  = lin1_layer.apply_fwd( $\mathbf{x}$ ,  $\boldsymbol{\alpha}$ )
12         $\mathbf{z}$  = sig_layer.apply_fwd( $\mathbf{a}$ )
13         $\mathbf{b}$  = lin2_layer.apply_fwd( $\mathbf{z}$ ,  $\boldsymbol{\beta}$ )
14         $\hat{\mathbf{y}}$  = soft_layer.apply_fwd( $\mathbf{b}$ )
15         $J$  = ce_layer.apply_fwd( $\mathbf{y}$ ,  $\hat{\mathbf{y}}$ )
16        return  $J$ .out_tensor
17
18    method backward(Tensor  $\mathbf{x}$ , Tensor  $\mathbf{y}$ , Tensor  $\boldsymbol{\alpha}$ , Tensor  $\boldsymbol{\beta}$ )
19        tape_bwd()
20        return lin1_layer.in_gradients[1], lin2_layer.in_gradients[1]
```

```
1 global tape = stack()
2
3 class Module:
4
5     method init()
6         out_tensor = null
7         out_gradient = Tensor(1)
8
9     method apply_fwd(List in_modules)
10        in_tensors = [x.out_tensor for x in in_modules]
11        out_tensor = forward(in_tensors)
12        tape.push(self)
13        return self
14
15    method apply_bwd():
16        in_gradients = backward(in_tensors, out_tensor, out_gradient)
17        for i in 1,..., len(in_modules):
18            in_modules[i].out_gradient += in_gradients[i]
19        return self
20
21    function tape_bwd():
22        while len(tape) > 0
23            m = tape.pop()
24            m.apply_bwd()
```

PyTorch

The same simple neural network we defined in pseudocode can also be defined in PyTorch.

```
1 # Define model
2 class NeuralNetwork(nn.Module):
3     def __init__(self):
4         super(NeuralNetwork, self).__init__()
5         self.flatten = nn.Flatten()
6         self.linear1 = nn.Linear(28*28, 512)
7         self.sigmoid = nn.Sigmoid()
8         self.linear2 = nn.Linear(512, 512)
9
10    def forward(self, x):
11        x = self.flatten(x)
12        a = self.linear1(x)
13        z = self.sigmoid(a)
14        b = self.linear2(z)
15        return b
16
17 # Take one step of SGD
18 def one_step_of_sgd(X, y):
19     model = NeuralNetwork().to(device)
20     loss_fn = nn.CrossEntropyLoss()
21     optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
22
23     # Compute prediction error
24     pred = model(X)
25     loss = loss_fn(pred, y)
26
27     # Backpropagation
28     optimizer.zero_grad()
29     loss.backward()
30     optimizer.step()
```

PyTorch

Q: Why don't we call `linear.forward()` in PyTorch?

A: This is just syntactic sugar. There's a special method in Python `__call__` that allows you to define what happens when you treat an object as if it were a function.

In other words, running the following:

```
linear(x)
```

is equivalent to running:

```
linear.__call__(x)
```

which in PyTorch is (nearly) the same as running:

```
linear.forward(x)
```

This is because PyTorch defines every Module's `__call__` method to be something like this:

```
def __call__(self):  
    self.forward()
```

PyTorch

Q: Why don't we pass in the parameters to a PyTorch Module?

A: This just makes your code cleaner.

In PyTorch, you store the parameters inside the Module and “mark” them as parameters that should contribute to the eventual gradient used by an optimizer

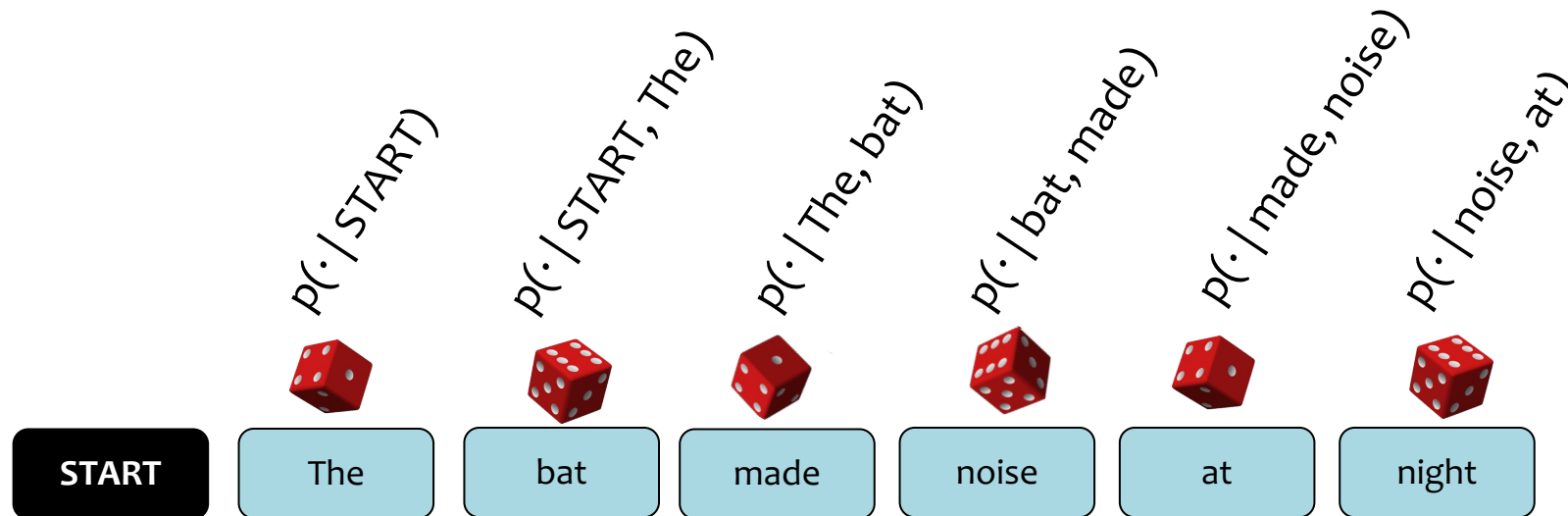
```
0  method forward(Tensor x , Tensor y , Tensor  $\alpha$  , Tensor  $\beta$ )
11      a =lin1_layer.apply_fwd(x,  $\alpha$ )
12      z =sig_layer.apply_fwd(a)
13      b =lin1_layer.apply_fwd(z,  $\beta$ )
14       $\hat{y}$  =soft_layer.apply_fwd(b)
15      J =ce_layer.apply_fwd(y,  $\hat{y}$ )
16      return J.out_tensor
```

```
7
10  def forward(self, x):
11      x = self.flatten(x)
12      a = self.linear1(x)
13      z = self.sigmoid(a)
14      b = self.linear2(z)
15      return b
```

BACKGROUND: N-GRAM LANGUAGE MODELS

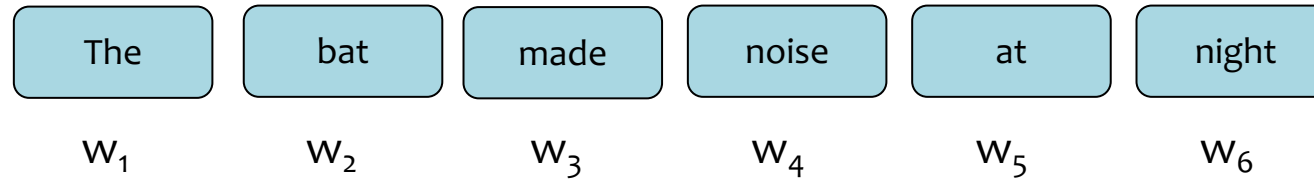
n-Gram Language Model

- Goal: Generate realistic looking sentences in a human language
- Key Idea: condition on the last $n-1$ words to sample the n^{th} word



The Chain Rule of Probability

Question: How can we **define** a probability distribution over a sequence of length T?

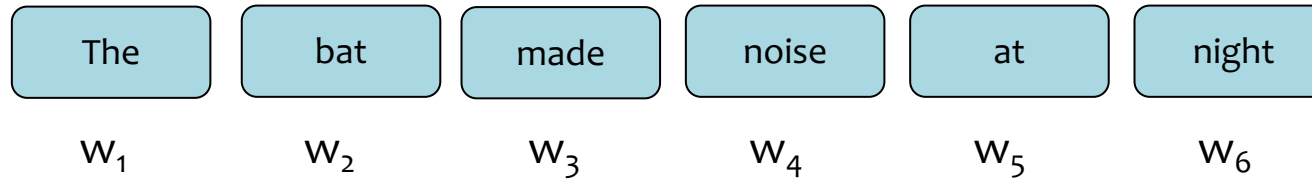


$$p(w_1, w_2, w_3, \dots, w_6) =$$

$p(w_1)$
 $p(w_2 | w_1)$
 $p(w_3 | w_2, w_1)$
 $p(w_4 | w_3, w_2, w_1)$
 $p(w_5 | w_4, w_3, w_2, w_1)$
 $p(w_6 | w_5, w_4, w_3, w_2, w_1)$

The Chain Rule of Probability

Question: How can we **define** a probability distribution over a sequence of length T?



Chain rule of probability:
$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid w_{t-1}, \dots, w_1)$$

$$p(w_1, w_2, w_3, \dots, w_6) =$$

$$p(w_1)$$

$$p(w_2 \mid w_1)$$

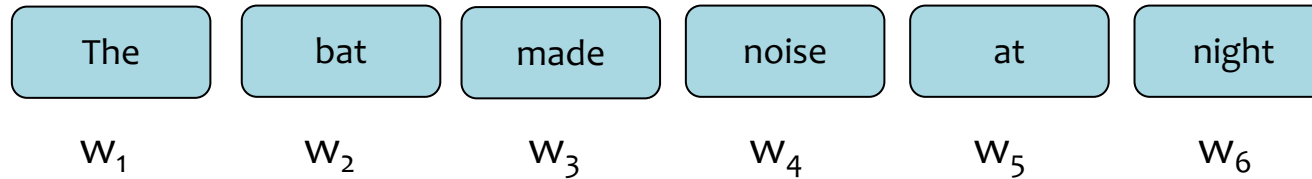
Note: This is called the chain **rule** because it is **always** true for every probability distribution

$$p(w_1)$$

$$p(w_6 \mid w_5, w_4, w_3, w_2, w_1)$$

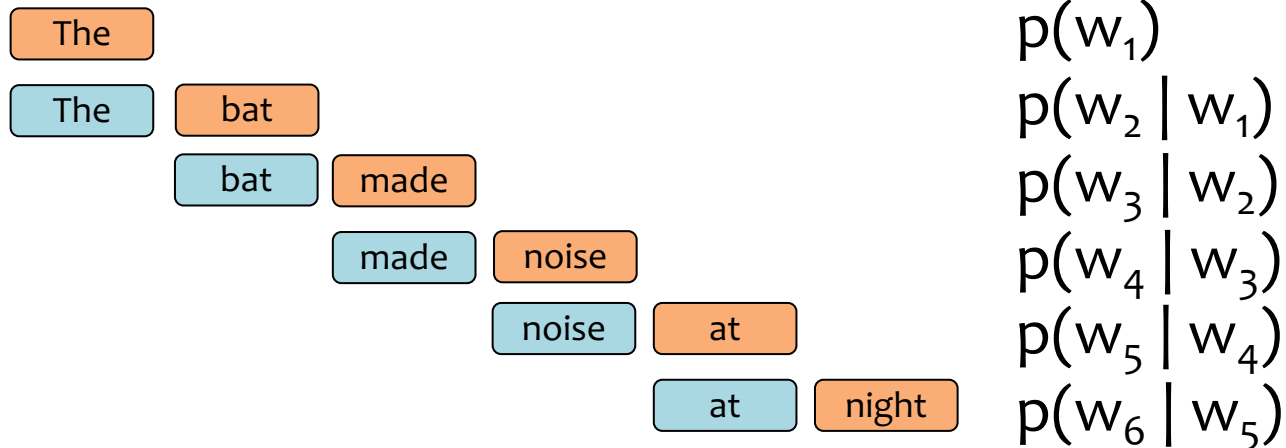
n-Gram Language Model

Question: How can we **define** a probability distribution over a sequence of length T?



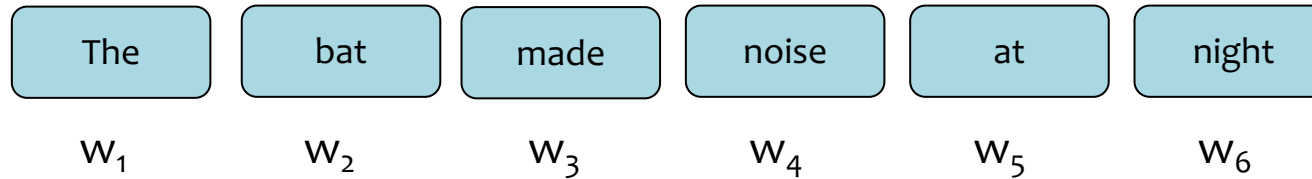
n-Gram Model (n=2)
$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid w_{t-1})$$

$$p(w_1, w_2, w_3, \dots, w_6) =$$



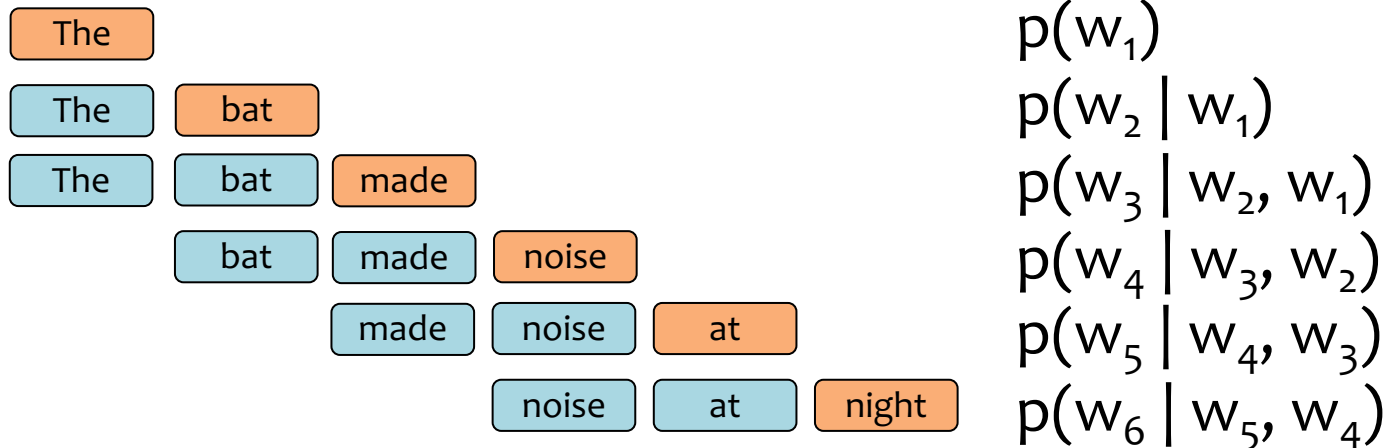
n-Gram Language Model

Question: How can we **define** a probability distribution over a sequence of length T?



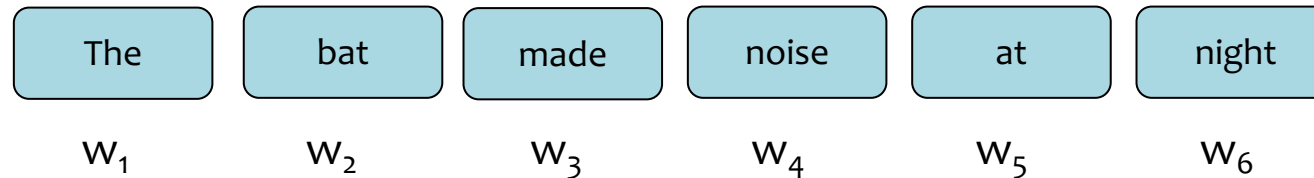
n-Gram Model (n=3)
$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid w_{t-1}, w_{t-2})$$

$$p(w_1, w_2, w_3, \dots, w_6) =$$



n-Gram Language Model

Question: How can we **define** a probability distribution over a sequence of length T?



n-Gram Model (n=3)
$$p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid w_{t-1}, w_{t-2})$$

$p(w_1, w_2, w_3, \dots, w_6) =$

The

The

The

$p(w_1)$

$p(w_2 \mid w_1)$

Note: This is called a **model** because we made some **assumptions** about how many previous words to condition on (i.e. only n-1 words)

Learning an n-Gram Model

Question: How do we **learn** the probabilities for the n-Gram Model?

$$p(w_t \mid w_{t-2} = \text{The}, w_{t-1} = \text{bat})$$



w_t	$p(\cdot \mid \cdot, \cdot)$
ate	0.015
...	
flies	0.046
...	
zebra	0.000

$$p(w_t \mid w_{t-2} = \text{made}, w_{t-1} = \text{noise})$$



w_t	$p(\cdot \mid \cdot, \cdot)$
at	0.020
...	
pollution	0.030
...	
zebra	0.000

$$p(w_t \mid w_{t-2} = \text{cows}, w_{t-1} = \text{eat})$$



w_t	$p(\cdot \mid \cdot, \cdot)$
corn	0.420
...	
grass	0.510
...	
zebra	0.000

Learning an n-Gram Model

Question: How do we **learn** the probabilities for the n-Gram Model?

Answer: From data! Just **count** n-gram frequencies

... the **cows eat grass**...
... our **cows eat hay** daily...
... factory-farm **cows eat corn**...
... on an organic farm, **cows eat hay** and...
... do your **cows eat grass** or corn?...
... what do **cows eat** if they have...
... **cows eat corn** when there is no...
... which **cows eat which** foods depends...
... if **cows eat grass**...
... when **cows eat corn** their stomachs...
... should we let **cows eat corn**?...

$$p(w_t \mid w_{t-2} = \text{cows}, w_{t-1} = \text{eat})$$



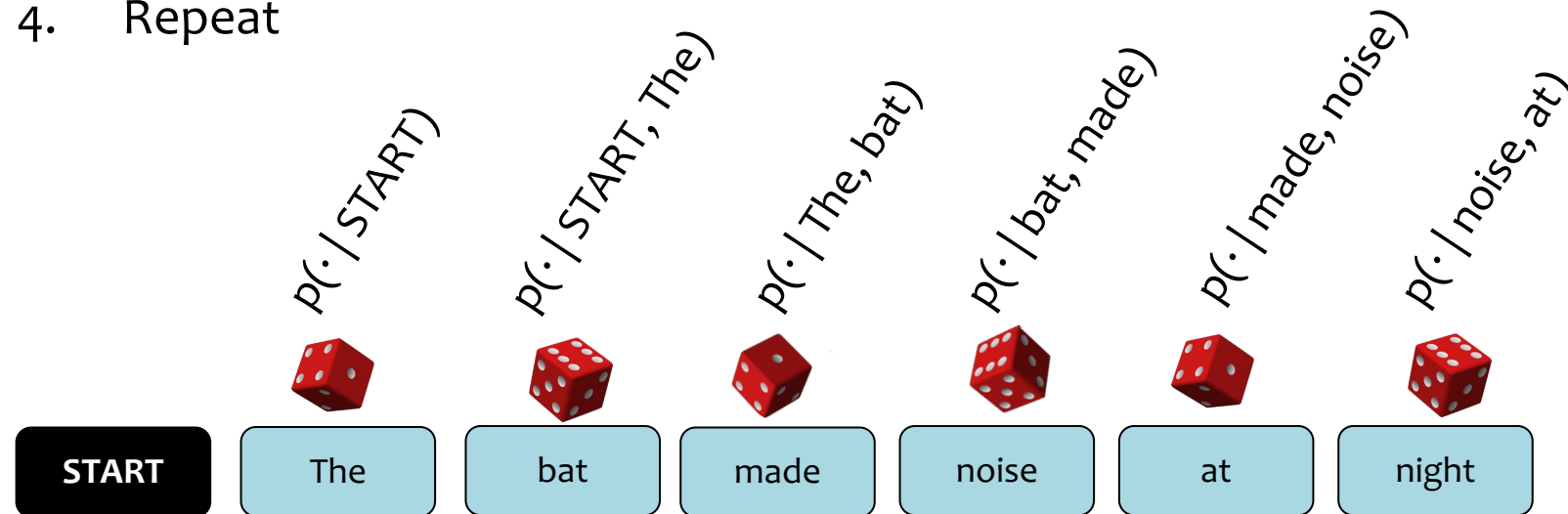
w_t	$p(\cdot \mid \cdot, \cdot)$
corn	4/11
grass	3/11
hay	2/11
if	1/11
which	1/11

Sampling from a Language Model

Question: How do we sample from a Language Model?

Answer:

1. Treat each probability distribution like a (50k-sided) weighted die
2. Pick the die corresponding to $p(w_t | w_{t-2}, w_{t-1})$
3. Roll that die and generate whichever word w_t lands face up
4. Repeat



Sampling from a Language Model

Question: How do we sample from a Language Model?

Answer:

1. Treat each probability distribution like a (50k-sided) weighted die
2. Pick the die corresponding to $p(w_t | w_{t-2}, w_{t-1})$
3. Roll that die and generate whichever word w_t lands face up
4. Repeat

Training Data (Shakespeare)	5-Gram Model
I tell you, friends, most charitable care ave the patricians of you. For your wants, Your suffering in this dearth, you may as well Strike at the heaven with your staves as lift them Against the Roman state, whose course will on The way it takes, cracking ten thousand curbs Of more strong link asunder than can ever Appear in your impediment. For the dearth, The gods, not the patricians, make it, and Your knees to them, not arms, must help.	Approacheth, denay. dungy Thither! Julius think: grant,--0 Yead linens, sheep's Ancient, Agreed: Petrarch plaguy Resolved pear! observingly honourest adulteries wherever scabbard guess; affirmation--his monsieur; died. jealousy, chequins me. Daphne building. weakness: sun- rise, cannot stays carry't, unpurposed. prophet-like drink; back-return 'gainst surmise Bridget ships? wane; interim? She's striving wet;

RECURRENT NEURAL NETWORK (RNN) LANGUAGE MODELS

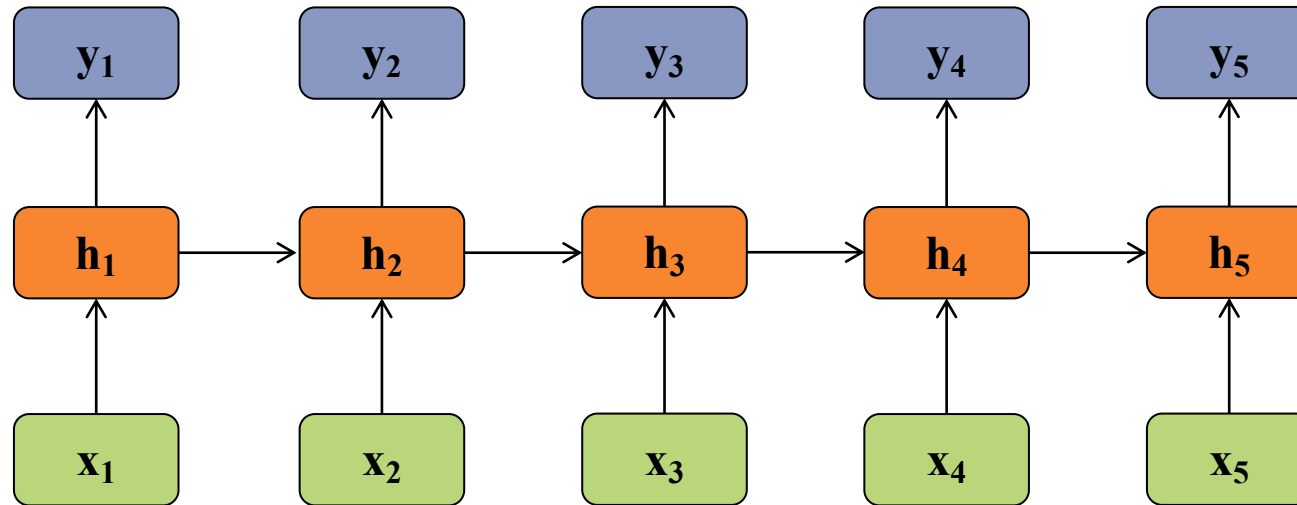
Recurrent Neural Networks (RNNs)

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

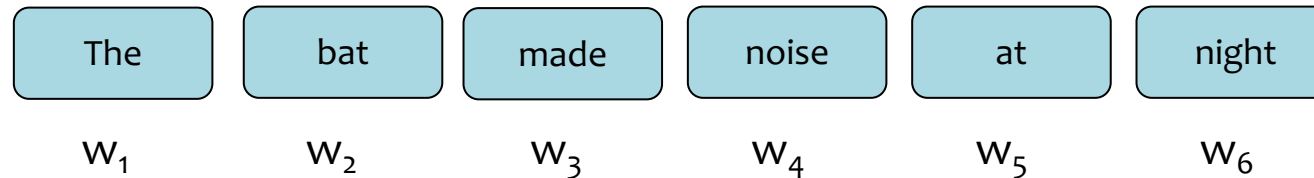
$$y_t = W_{hy}h_t + b_y$$



The Chain Rule of Probability

Recall...

Question: How can we **define** a probability distribution over a sequence of length T?



Chain rule of probability: $p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid w_{t-1}, \dots, w_1)$

$p(w_1, w_2, w_3, \dots, w_6) =$

$p(w_1)$

$p(w_2 \mid w_1)$

Note: This is called the chain **rule** because it is **always** true for every probability distribution

$p(w_1)$

$p(w_6 \mid w_5, w_4, w_3, w_2, w_1)$

RNN Language Model

$$\text{RNN Language Model: } p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t \mid f_{\theta}(w_{t-1}, \dots, w_1))$$

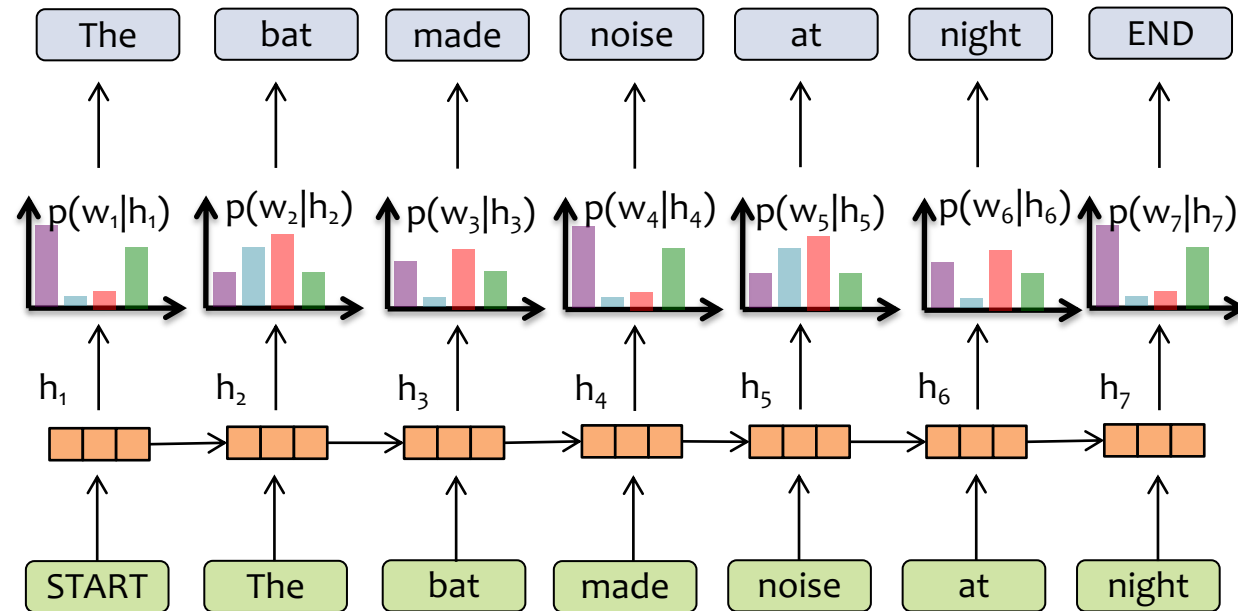
$$p(w_1, w_2, w_3, \dots, w_6) =$$

The						$p(w_1)$
The	bat					$p(w_2 \mid f_{\theta}(w_1))$
The	bat	made				$p(w_3 \mid f_{\theta}(w_2, w_1))$
The	bat	made	noise			$p(w_4 \mid f_{\theta}(w_3, w_2, w_1))$
The	bat	made	noise	at		$p(w_5 \mid f_{\theta}(w_4, w_3, w_2, w_1))$
The	bat	made	noise	at	night	$p(w_6 \mid f_{\theta}(w_5, w_4, w_3, w_2, w_1))$

Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t \mid f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector

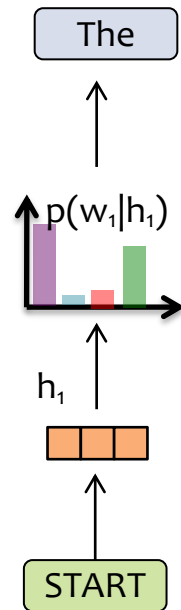
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

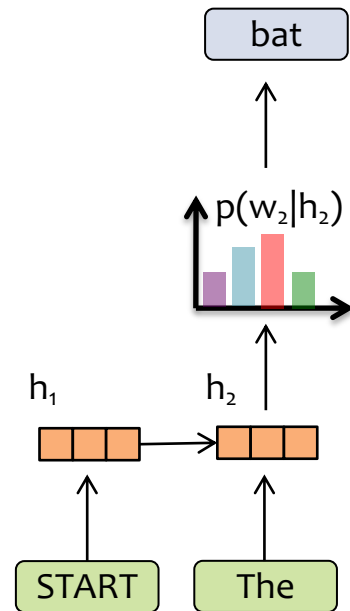
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t \mid f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

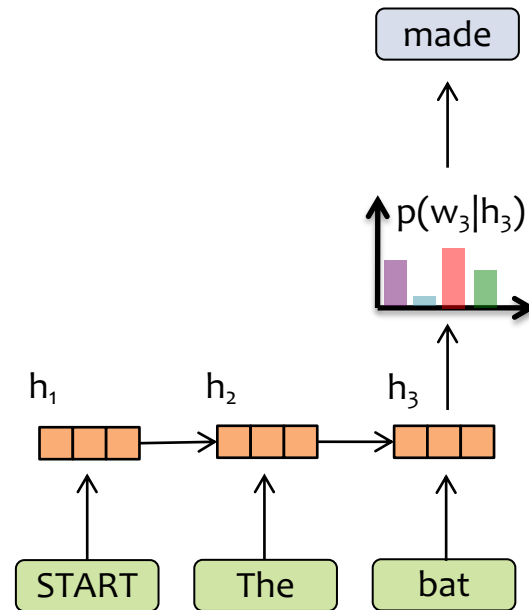
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

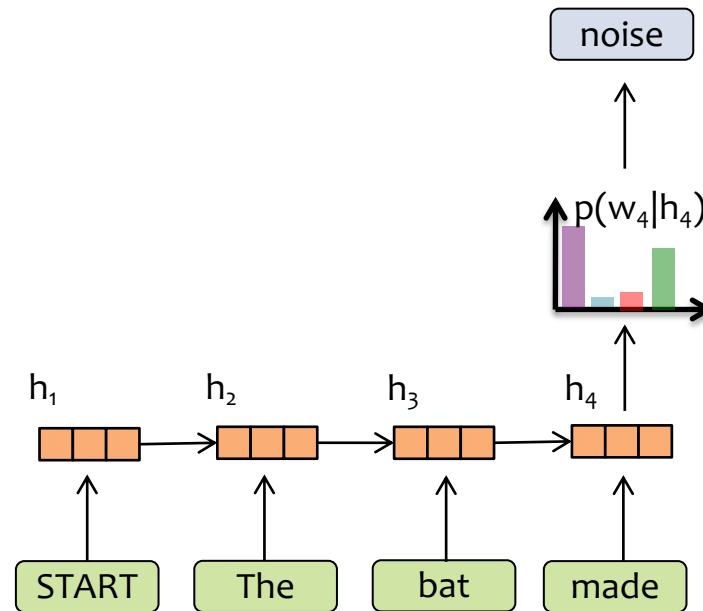
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

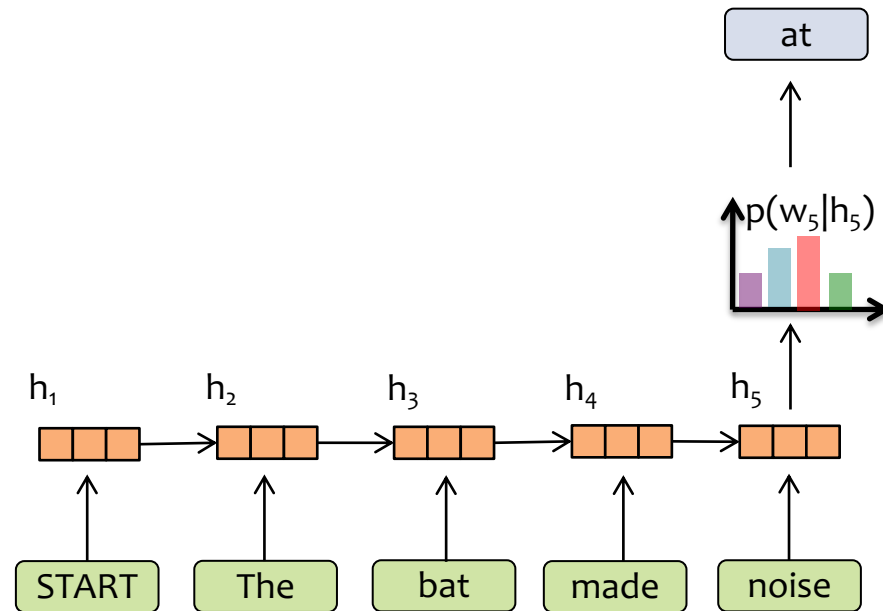
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

RNN Language Model



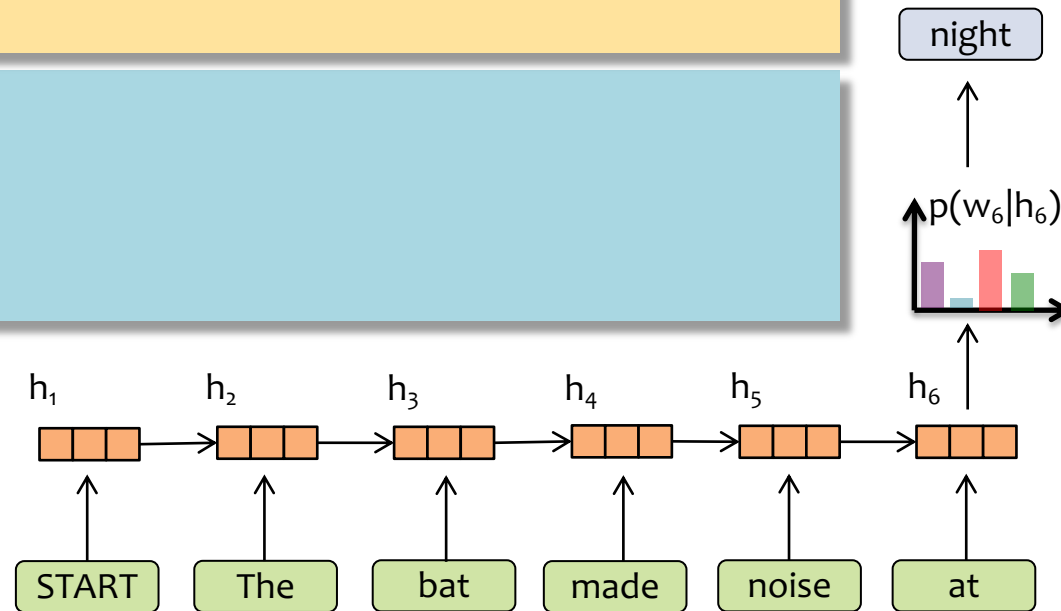
Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

RNN Language Model

Question: How can we create a distribution $p(w_t|h_t)$ from h_t ?

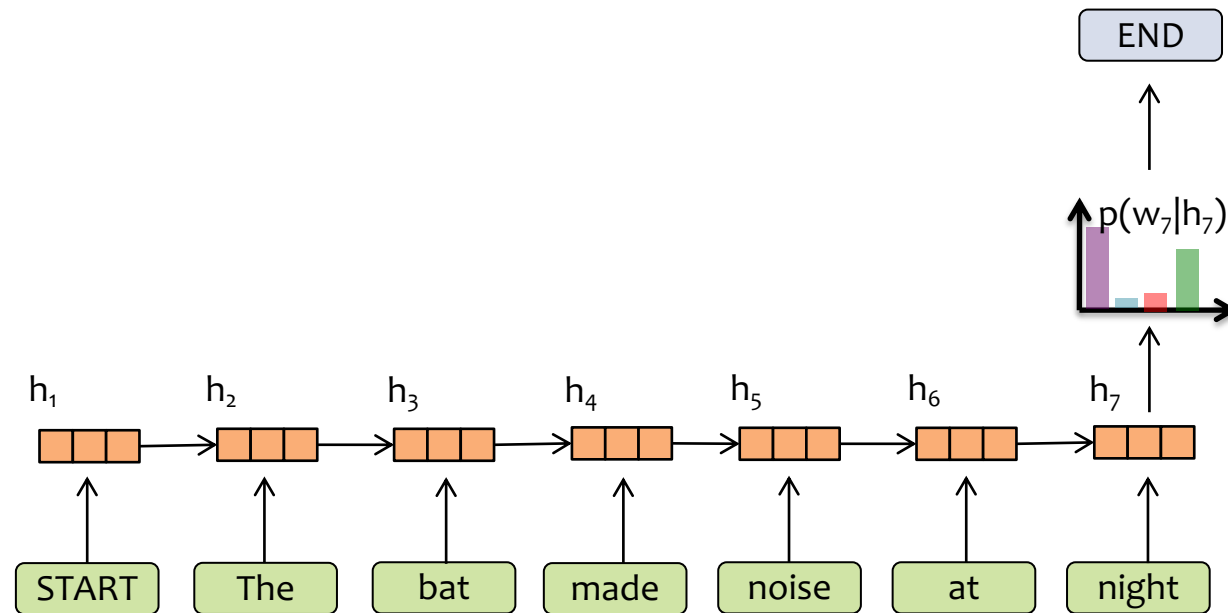
Answer:



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

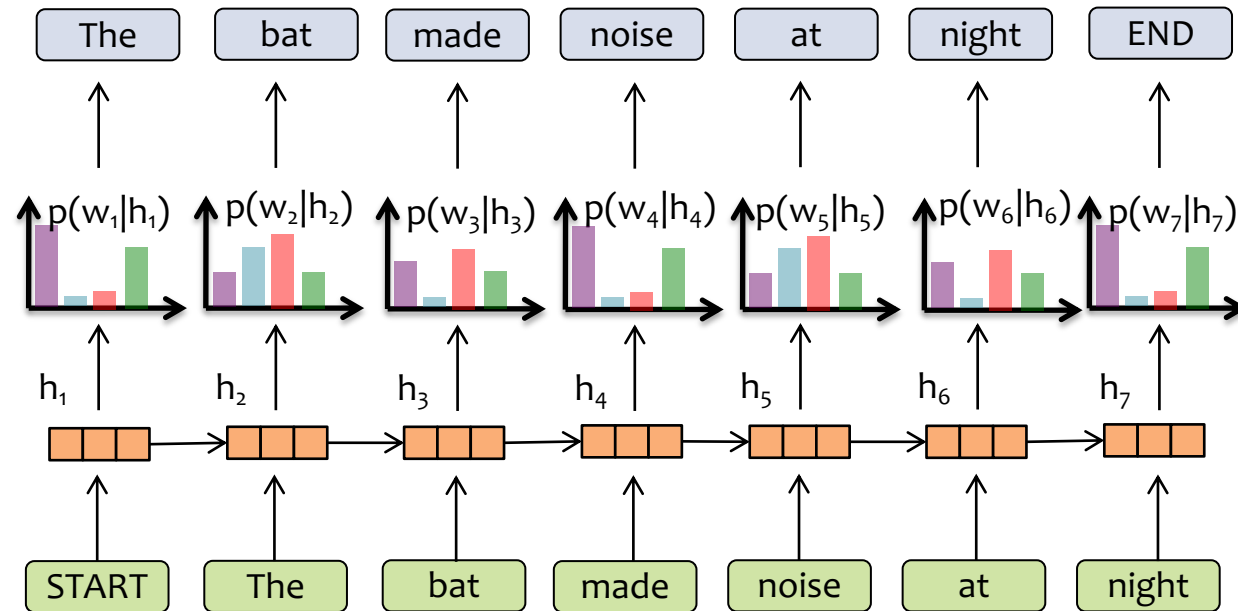
RNN Language Model



Key Idea:

- (1) convert all previous words to a **fixed length vector**
- (2) define distribution $p(w_t | f_{\theta}(w_{t-1}, \dots, w_1))$ that conditions on the vector $\mathbf{h}_t = f_{\theta}(w_{t-1}, \dots, w_1)$

RNN Language Model



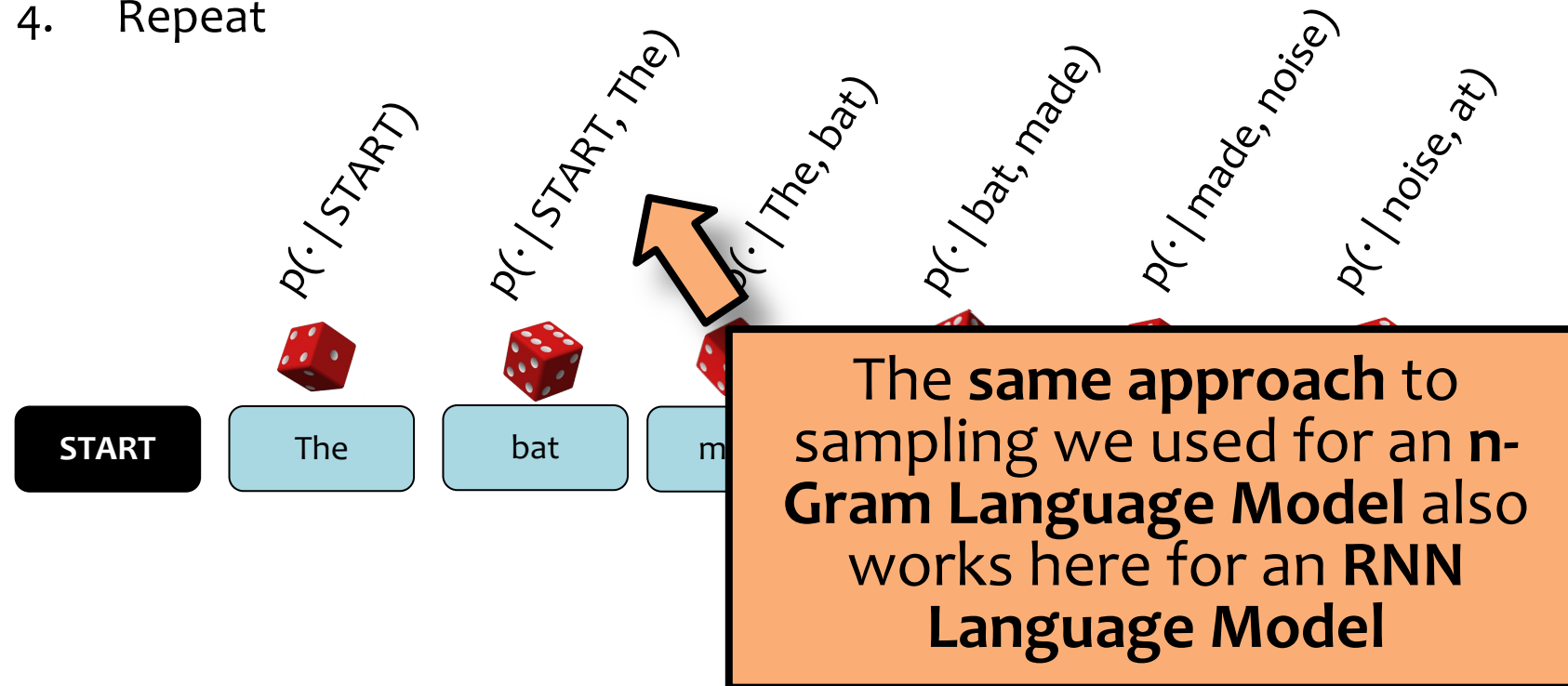
$$p(w_1, w_2, w_3, \dots, w_T) = p(w_1 | h_1) p(w_2 | h_2) \dots p(w_T | h_T)$$

Sampling from a Language Model

Question: How do we sample from a Language Model?

Answer:

1. Treat each probability distribution like a (50k-sided) weighted die
2. Pick the die corresponding to $p(w_t | w_{t-2}, w_{t-1})$
3. Roll that die and generate whichever word w_t lands face up
4. Repeat



Sampling from an RNN-LM

??

VIOLA: Why, Salisbury must find his flesh and thought
That which I am not apt, not a man and in fire, To show
the reining of the raven and the wars To grace my hand
reproach within, and not a fair are hand, That Caesar and
my goodly father's world; When I was heaven of
presence and our fleets, We spare with hours, but cut thy
council I am great, Murdered and by thy m
there My power to give thee but so much
service in the noble bondman here, Would
her wine.

KING LEAR: O, if you were a feeble sight, the courtesy of
your law, Your sight and several breath, will wear the
gods With his heads, and my hands are wonder'd at the
deeds, So drop upon your lordship's head, and your
opinion Shall be against your honour.

??

CHARLES: Marry, do I, sir; and I came to acquaint you
with a matter. I am given, sir, secretly to understand that
your younger brother Orlando hath a disposition to come
in disguised against me to try a fall. To-morrow, sir, I
wrestle for my credit; and he that escapes me without
some broken limb shall acquit him well. Your brother is
tender; and, for your love, I would be
as I must, for my own honour, if he
more, out of my love to you, I came hither
to acquaint you withal, that either you might stay him
from his intended or brook such disgrace well as he
shall run into, in that is a thing of his own search and
altogether against my will.

TOUCHSTONE: For my part, I had rather bear with you
than bear you; yet I should bear no cross if I did bear you,
for I think you have no money in your purse.

Which is the real
Shakespeare?!

Sampling from an RNN-LM

Shakespeare's As You Like It

VIOLA: Why, Salisbury must find his flesh and thought
That which I am not aps, not a man and in fire, To show
the reining of the raven and the wars To grace my hand
reproach within, and not a fair are hand, That Caesar and
my goodly father's world; When I was heaven of
presence and our fleets, We spare with hours, but cut thy
council I am great, Murdered and by thy master's ready
there My power to give thee but so much as hell: Some
service in the noble bondman here, Would show him to
her wine.

KING LEAR: O, if you were a feeble sight, the courtesy of
your law, Your sight and several breath, will wear the
gods With his heads, and my hands are wonder'd at the
deeds, So drop upon your lordship's head, and your
opinion Shall be against your honour.

RNN-LM Sample

CHARLES: Marry, do I, sir; and I came to acquaint you
with a matter. I am given, sir, secretly to understand that
your younger brother Orlando hath a disposition to come
in disguised against me to try a fall. To-morrow, sir, I
wrestle for my credit; and he that escapes me without
some broken limb shall acquit him well. Your brother is
but young and tender; and, for your love, I would be
loath to foil him, as I must, for my own honour, if he
come in: therefore, out of my love to you, I came hither
to acquaint you withal, that either you might stay him
from his intendment or brook such disgrace well as he
shall run into, in that it is a thing of his own search and
altogether against my will.

TOUCHSTONE: For my part, I had rather bear with you
than bear you; yet I should bear no cross if I did bear you,
for I think you have no money in your purse.

Sampling from an RNN-LM

RNN-LM Sample

VIOLA: Why, Salisbury must find his flesh and thought
That which I am not aps, not a man and in fire, To show
the reining of the raven and the wars To grace my hand
reproach within, and not a fair are hand, That Caesar and
my goodly father's world; When I was heaven of
presence and our fleets, We spare with hours, but cut thy
council I am great, Murdered and by thy master's ready
there My power to give thee but so much as hell: Some
service in the noble bondman here, Would show him to
her wine.

KING LEAR: O, if you were a feeble sight, the courtesy of
your law, Your sight and several breath, will wear the
gods With his heads, and my hands are wonder'd at the
deeds, So drop upon your lordship's head, and your
opinion Shall be against your honour.

Shakespeare's As You Like It

CHARLES: Marry, do I, sir; and I came to acquaint you
with a matter. I am given, sir, secretly to understand that
your younger brother Orlando hath a disposition to come
in disguised against me to try a fall. To-morrow, sir, I
wrestle for my credit; and he that escapes me without
some broken limb shall acquit him well. Your brother is
but young and tender; and, for your love, I would be
loath to foil him, as I must, for my own honour, if he
come in: therefore, out of my love to you, I came hither
to acquaint you withal, that either you might stay him
from his intendment or brook such disgrace well as he
shall run into, in that it is a thing of his own search and
altogether against my will.

TOUCHSTONE: For my part, I had rather bear with you
than bear you; yet I should bear no cross if I did bear you,
for I think you have no money in your purse.

Sampling from an RNN-LM

??

VIOLA: Why, Salisbury must find his flesh and thought
That which I am not apt, not a man and in fire, To show
the reining of the raven and the wars To grace my hand
reproach within, and not a fair are hand, That Caesar and
my goodly father's world; When I was heaven of
presence and our fleets, We spare with hours, but cut thy
council I am great, Murdered and by thy m
there My power to give thee but so much
service in the noble bondman here, Would
her wine.

KING LEAR: O, if you were a feeble sight, the courtesy of
your law, Your sight and several breath, will wear the
gods With his heads, and my hands are wonder'd at the
deeds, So drop upon your lordship's head, and your
opinion Shall be against your honour.

??

CHARLES: Marry, do I, sir; and I came to acquaint you
with a matter. I am given, sir, secretly to understand that
your younger brother Orlando hath a disposition to come
in disguised against me to try a fall. To-morrow, sir, I
wrestle for my credit; and he that escapes me without
some broken limb shall acquit him well. Your brother is
tender; and, for your love, I would be
as I must, for my own honour, if he
more, out of my love to you, I came hither
to acquaint you withal, that either you might stay him
from his intended or brook such disgrace well as he
shall run into, in that is a thing of his own search and
altogether against my will.

TOUCHSTONE: For my part, I had rather bear with you
than bear you; yet I should bear no cross if I did bear you,
for I think you have no money in your purse.

Which is the real
Shakespeare?!